One GEMM, Many Subgames: A Predictive Cost Model and an End-to-End Wall-Clock Win for Iterated Resolving in Imperfect-Information Games

Max Jingwei Zhang*†‡[1,2]

[1]Ontario Teachers' Pension Plan
[2]University of Toronto, Faculty of Applied Science and Engineering

June 10, 2026

Keywords: Imperfect-Information Games; Counterfactual Regret Minimization; Iterated Resolving; Public-Belief Search; GPU Acceleration; Batched Solvers

Abstract

Depth-limited search with learned value functions in imperfect-information games repeatedly solves many public-belief subgames. This work isolates a systems lever in that loop: at a fixed public cut, resolving generates same-topology subgames that differ only in entry beliefs and solver state. **Population fusion** compiles that shared topology once into a level-grouped structure-of-arrays and runs each finite-iteration CFR+ update across the whole population as batched tensor operations. The formal timing model is a floor/slope model, $C_{\text{seq}}(N) = a_s + b_s(N-1)$ and $C_{\text{fus}}(N) = a_f + b_f(N-1)$; the familiar $\min(N, 1/\varphi)$ expression is the launch-bound envelope, not a universal plateau predictor. On launch-bound Goofspiel-4, the tool-symmetric `torch.compile` comparison gives 11.36× inner and 7.38× total-wall speedup over 5 matched CUDA seeds, against a 7.39× Amdahl ceiling and with no gross divergence under the paper's predeclared descriptive band check. The eager-dispatch isolate gives 11.79× inner and 9.99× total-wall speedup. On saturated Goofspiel-5, a quiet-host paired run gives 2.40× inner and 2.00× total timing speedup; this is saturated-regime timing-transfer evidence, with the full statistical CUDA non-divergence run still pending. The contribution is validated population-level cost-per-solve amortization for iterated resolving, not lower exploitability or poker-bot strength.

1 Introduction

Depth-limited resolving makes search usable in large imperfect-information games: solve a small lookahead game, use a value function at the depth limit, and repeat. This is the template behind DeepStack's continual re-solving with learned counterfactual values [43], ReBeL's public-belief search and reinforcement learning [12], and Student of Games' search-learning-game-theoretic unification [49]. In training, however, the template creates a hidden systems cost. The agent

*Correspondence: `maxjingwei.zhang@mail.utoronto.ca`.

†Work performed in Ontario Teachers' Pension Plan, Quantitative Modeling and Programming.

‡The views expressed in this paper are solely those of the author and do not necessarily reflect the views, policies, positions, investment opinions, or risk-management practices of Ontario Teachers' Pension Plan or its affiliates.

repeatedly asks a solver to produce value targets for many public-belief subgames: small continuation games rooted at the same public history but conditioned on different beliefs over hidden information. Those beliefs matter because, unlike in chess or Go, an imperfect-information position does not have a single value unless it is tied to what both players might privately hold and believe. The public history plus those belief distributions is the *public belief state* (PBS), and naively truncating search without belief-conditioned leaf values yields exploitable strategies [8].

The important systems fact is that one resolving sweep does not produce one arbitrary subgame at a time. It produces a *subgame population*. At a fixed public cut, each public outcome, or key, roots a continuation subgame. In the loop studied here these continuation trees share topology; what changes from key to key is the data flowing through the tree: the two players' entry beliefs, or *ranges*, and the solver's per-subgame state. Running the same GPU solver separately for every key therefore repeats the same short-kernel launch sequence many times. For small subgames this is launch-bound: the GPU can be underfilled even though the training loop spends almost all of its time resolving.

This paper studies that repeated target-generation step as a cost-per-solve problem. The wall-clock cost of the resolving phase factors as

$$C = \underbrace{N_{\text{solves}}}_{\text{how many subgames}} \times \underbrace{T}_{\text{CFR iterations per solve}} \times \underbrace{c_{\text{solve}}}_{\text{wall-clock per solve-iteration}}, \tag{1}$$

an accounting identity in which N_{solves} counts subgame solves over the run, T is the per-solve CFR+ iteration budget, and c_{solve} is wall-clock per solve-iteration. The loop's remaining phases — network training and measurement — sit outside this factor and are handled by the Amdahl analysis of §6.2. In our Goofspiel-4 (G4) operating point, the inner re-solve accounts for 98.4% of baseline loop wall-clock (§3).

One factor of Eq. (1) is under-isolated. Existing work makes resolving practical by changing how many subgames are solved, how deep or large each subgame is, how many iterations are needed, or how one tree's CFR computation is parallelized (§3, §7). Recent GPU-CFR work, including NoRegret-style linear-algebra CFR [24, 25] and real-time HUNL-oriented parallel CFR [36], is primarily a within-tree or single-solve systems story. The missing systems unit here is the resolving sweep: amortize c_{solve} across the same-topology subgame population generated by one sweep, not only inside one member of that population.

The mechanism is across-key population fusion. Many same-shaped small problems differing only in data are the workload that batched BLAS [1, 2] and FlashAttention-style kernel engineering [16] teach us to exploit: fuse them, so the device performs the same arithmetic under far fewer kernel launches. We call the resulting mechanism **population fusion**: compile the shared subgame topology once into a level-grouped structure-of-arrays (SoA), fold the public-key population into the tensor dimensions, and execute each finite-budget CFR+ iteration for all keys as one fixed sequence of batched tensor ops (Figure 1; §4). The inner solver is CFR+ [54, 55, 5], descended from CFR [59]. The "GEMM" in the title names the systems pattern — many same-shaped solves turned into one batched tensor workload — not literal matrix multiplication at every step. The fusion axis is K, the number of public keys at the cut; the within-key private-hand/range dimension is classical range-vector CFR [22, 23] and is present in both timing arms. The fused solver is verified to match the per-tree finite-iteration CFR+ outputs under the gates of §4.5.

A predictive cost model. When does fusion pay? A controlled sweep gives a floor/slope answer, $C_{\text{seq}}(N) = a_s + b_s(N-1)$ and $C_{\text{fus}}(N) = a_f + b_f(N-1)$: speedup is near-linear when the fused marginal slope is near zero, and contracts to a slope ratio once one subgame already fills the device (§6.1). The simplified $\min(N, 1/\varphi)$ form is useful as the launch-bound envelope and regime

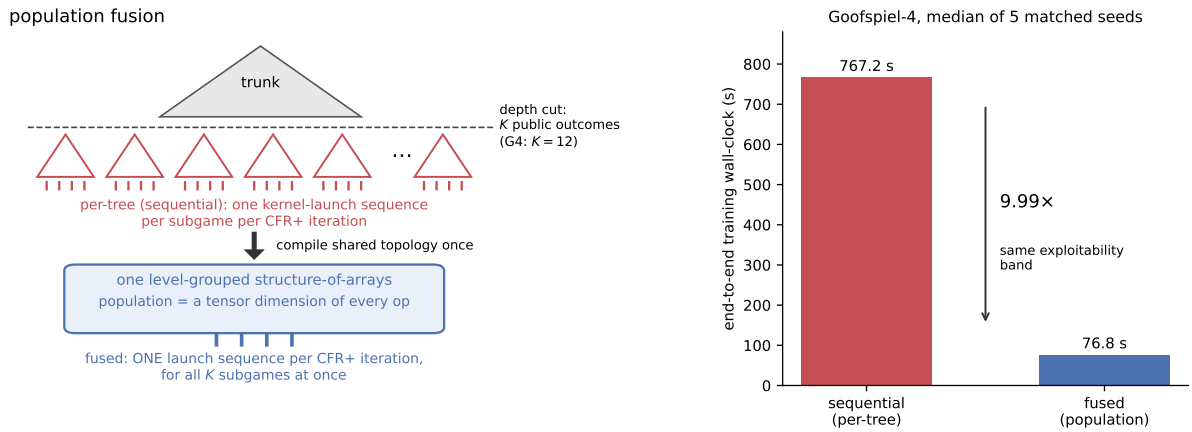

Figure 1: Population fusion, and what it buys end to end. *Left:* at the depth cut, the *trunk* — the above-cut portion of the game, which the agent solves directly and whose leaf values the subgame population provides — has K public outcomes, each rooting a same-topology subgame (G4: $K = 12$); solved per-tree, each subgame costs its own kernel-launch sequence per CFR+ iteration. Population fusion compiles the shared topology once into a level-grouped structure-of-arrays and folds the population into a tensor dimension of every op, so one launch sequence per CFR+ iteration solves all K subgames. *Right:* end-to-end Goofspiel-4 training wall-clock, median of 5 matched seeds: sequential 767.2 s vs fused 76.8 s — 9.99×, with no gross divergence under the descriptive band check (§6.2). The tool-symmetric compile-vs-compile check gives 7.38× total-wall speedup (§6.5).

intuition, while a two-point marginal-cost probe classifies which regime a workload is in before committing to fusion.

The win survives a real training loop. Microbenchmark speedups routinely evaporate inside end-to-end systems; this one survives because the loop is dominated by the phase we fused. On Goofspiel-4 — small subgames, the launch-bound regime — the fused inner re-solve is 11.79× faster than the same kernel applied per tree, against the 11.7× the microbenchmark sweep predicted ex ante, and total training wall-clock improves 9.99× against the 10.09× Amdahl ceiling implied by the measured inner fraction (5 matched seeds; §6.2). A tool-symmetric `torch.compile` comparison gives 11.36× inner and 7.38× total-wall speedup (§6.5), so the result is not merely an eager-dispatch artifact; CUDA Graphs are the rival launch-amortization lever considered in that analysis [44]. The speedup buys no lower-exploitability claim: the arms pass the paper's broad descriptive band check — fused median 0.0402, sequential 0.0469, where for scale uniform random play scores ≈1.42 on this game and 0 is exact Nash. On Goofspiel-5, where one subgame already fills the device, the loop realizes the predicted saturated speedups: 2.40× inner and 2.00× total (3 matched seeds; §6.4). As a directional calibration only, a search-free NFSP baseline trained on the same game remains near uniform after 600 s — a budget roughly eight times the fused loop's entire run (§6.3).

Scope. The G4 training claims are *equal-band-faster* — the same broad descriptive final-exploitability band in less wall-clock — never lower exploitability, and never strength against poker bots. The G5 paired run is a saturated-regime timing-transfer result with CPU/single-solve parity; the full statistical CUDA non-divergence run remains future work. The headline eager comparisons are same-substrate, against the same kernel run per-tree; the measured launch-amortization ablation and the G4 compile-vs-compile end-to-end check show that the advantage survives tool symmetry (§4.6, §6.5). The demonstrated loop is a 2-level, single-cut instance, on one game family and one device; §8 states every limitation in full.

Contributions

- **Identification of the population structure as a cost-per-solve lever.** Train-time iterated resolving structurally generates, at every iteration, a population of K same-topology depth-limited PBS subgames differing only in entry beliefs, and amortizing cost-per-solve across that population is, to our knowledge, not isolated and validated end to end in published work (§3, §7). The lever is orthogonal in form to the four claimed levers of Eq. (1); composition with them is untested (§8).
- **A parity-verified fused solver.** Population fusion compiles the shared topology once into a level-grouped, flat structure-of-arrays CFR+ and runs each iteration of *all* subgames as one fixed sequence of batched tensor ops, recovering classical range-vector semantics across heterogeneous decision/chance/variable-action structure and across public keys. It matches the per-tree finite-iteration CFR+ average strategies and leaf values, in the verified sense defined by the gates of §5 (Proposition 1, §4.5).
- **A predictive floor/slope cost model with a two-point regime probe.** The model is $C_{\text{seq}}(N) = a_s + b_s(N-1)$ and $C_{\text{fus}}(N) = a_f + b_f(N-1)$: near-linear while the fused marginal slope is near zero, and slope-ratio-limited once one subgame fills the device. The $\min(N, 1/\varphi)$ form remains the launch-bound envelope and regime intuition (§6.1). The microbenchmark prediction transfers to the live training loop at both ends (§6.2, §6.4).
- **An Amdahl-surviving end-to-end training win.** In a tabula-rasa self-play resolving loop on Goofspiel-4 (5 matched seeds, one configuration fixed in advance), tool-symmetric compile-vs-compile fusion gives 11.36× inner and 7.38× total-wall speedup against a 7.39× Amdahl ceiling. The eager-dispatch isolate gives 11.79× inner and 9.99× total wall-clock speedup (fused median 76.8 s vs sequential 767.2 s) against a 10.09× Amdahl ceiling. The sequential arm is a same-kernel per-tree GPU baseline analogous to within-tree GPU-CFR [24]; it is not Kim's implementation or NoRegret. Both comparisons make no lower-exploitability claim and are gated by the broad descriptive band check (§6.2, §6.5).
- **A finding about evaluating batched GPU game solvers.** CUDA atomics compound through CFR training into per-seed final-exploitability divergences far larger than any reasonable per-seed tolerance, so naive per-seed equivalence checks of batched GPU CFR through a training loop will spuriously fail. The remedy is part of the method: gate algorithmic equivalence on CPU, where the two arms are exactly the same algorithm, and use GPU runs for timing and band-level comparison only (§5, §6.2).

Table 1: Novelty boundary. The contribution is the validated conjunction in the last row, not batching or GPU-CFR in isolation.

Line of work	What it parallelizes or amortizes	Boundary relative to this paper
GPU-CFR / NoRegret [24, 25]	One extensive-form tree's CFR iteration, reframed as linear algebra	Our sequential arm is the same-kernel per-tree analogue; the new axis is a population of same-topology subgames from one resolving sweep.
Real-Time Parallel CFR [36]	One large HUNL depth-limited solve through CPU/GPU pipeline stages, pruning, abstraction, and GPU leaf evaluation	A single-solve real-time pipeline; orthogonal to same-topology population fusion.
DeepStack/ReBeL-style resolving [43, 12]	Learned value leaves and public-belief search/resolving	Supplies the workload family; solve-kernel population fusion is not isolated there.
Distributed subgame solving [5, 55]	Independent subgames over distributed workers	Task parallelism across workers, not one fused single-device kernel-launch sequence.
This work	All same-topology cut subgames in one resolving sweep	Parity-gated fused CFR+ iteration, predictive regime model, and end-to-end timing validation.

2 Background

This section teaches the three prerequisites the rest of the paper assumes: how CFR+ solves an imperfect-information game (§2.1), what depth-limited resolving and public belief states are and where the population of subgames arises (§2.2), and why many small GPU workloads are slow (§2.3). It closes with the paper's notation and project vocabulary. Readers fluent in CFR and GPU performance can skim directly to Table 3.

2.1 Extensive-form games, regret, and CFR+

A two-player zero-sum *extensive-form game* is a game tree with three kinds of nodes: decision nodes, where one player acts; chance nodes, where nature acts with known probabilities (a card is dealt, a prize is revealed); and terminals, which pay out. Hidden information is modeled by *information sets* (infosets): an infoset collects all the tree states a player cannot distinguish given what they have observed, and the player must act identically across them. A *strategy* σ assigns each infoset a probability distribution over its actions. A *Nash equilibrium* is a strategy profile in which neither player can gain by deviating; *exploitability* measures distance from it in plain terms — how much an optimal opponent could gain against the strategy. Zero means exact Nash; on Goofspiel-4, the running example introduced below, uniform random play scores ≈1.42. (The formal metric, NashConv, is defined in §5.)

Counterfactual Regret Minimization (CFR) [59] solves such games by iterated self-play over the regrets of actions. The *regret* of an action at an infoset is how much better that action would have done than the current strategy, in hindsight, weighted by the *counterfactual reach* — the probability that the opponent and chance alone would bring play to this infoset. (The player's own contribution to reaching the infoset is divided out; that is what "counterfactual" means.) *Regret matching* [19]

Table 2: Three toy CFR+ iterations at one infoset (**illustrative numbers only**).

iter	$R = (R_a, R_b)$	$\sigma \propto [R]_+$	$v(a), v(b)$	$v(\sigma)$	deltas $\times 0.5$	new R
1	(6.00, 2.00)	(0.75, 0.25)	1, 3	1.50	$(-0.25, +0.75)$	(5.75, 2.75)
2	(5.75, 2.75)	(0.68, 0.32)	2, -3	0.40	$(+0.80, -1.70)$	(6.55, 1.05)
3	(6.55, 1.05)	(0.86, 0.14)	1, -4	0.30	$(+0.35, -2.15)$	(6.90, **0.00**)

turns accumulated regrets into the next strategy: play each action with probability proportional to its positive cumulative regret — a clamped normalization. Every CFR variant shares one iteration structure: **(1)** regret-match the current strategy from the regret table; **(2)** a *forward pass* propagates reach probabilities from the root to the leaves; **(3)** a *backward pass* propagates expected values from the leaves to the root, depositing per-action regret contributions at each infoset; **(4)** update the cumulative-regret table and an average-strategy table. The convergence fact that drives everything: it is the *time-averaged* strategy — not the last iterate — that converges to a Nash equilibrium, at rate $O(1/\sqrt{T})$ in the iteration count T, which is why an average-strategy table S is carried alongside the regret table R. CFR+ [54] is the practical workhorse variant: it clamps cumulative regrets at zero (so an action that has done badly can re-enter quickly), alternates which player updates each iteration, and weights the average linearly toward later iterates [55, 14].

A worked mini-example. Table 2 runs three CFR+ iterations at a single infoset with two actions $\{a, b\}$, holding the counterfactual reach at 0.5. *All numbers in Table 2 are pedagogical toys, not measurements*; strategy entries are rounded to two decimals, and the rounded values are used in the next row's arithmetic so the table is self-contained (the average-strategy accumulation is omitted for compactness).

Read row 1: cumulative regrets $(6, 2)$ regret-match to $\sigma = (0.75, 0.25)$; this iteration's counterfactual action values are $v(a) = 1$ and $v(b) = 3$, so the infoset is worth $0.75 \cdot 1 + 0.25 \cdot 3 = 1.5$ under σ; each action's regret delta is its value minus the infoset value, scaled by the counterfactual reach 0.5, giving new cumulative regrets $(5.75, 2.75)$ and hence next strategy $(0.68, 0.32)$. In row 3, action b's running total goes negative ($1.05 - 2.15 = -1.10$) and CFR+ floors it at zero — the clamp — after which b is played with probability 0 but can re-enter as soon as it accumulates positive regret again. The four-step skeleton — (1) regret-match, (2) forward reach, (3) backward counterfactual values, (4) regret/average update — is exactly what §4.3 turns into four batched tensor operations.

2.2 Depth-limited resolving, public belief states, and where the population arises

Vocabulary first. *Public* information is observed by both players: revealed cards, bids, the board, the betting so far. *Private* information is a player's hidden holding. The *reach probability* of a state under a strategy profile is the product of the action probabilities along the path to it. A player's *range* at a public state is a probability vector over that player's possible private states — what an observer who saw only the public actions should believe the player holds. A *public belief state* (PBS) packages exactly this:

$$\beta = (s_{\text{pub}}, (r_1, r_2)),$$

a public state s_{pub} together with each player i's range r_i over their private states consistent with s_{pub} [12]. In poker, β is the community cards and betting history plus each player's distribution over hole cards. In our running example it is the following.

Running example: Goofspiel. In Goofspiel-k [45] each player holds a hand of k cards, valued 1 to k, used as sealed bids (Goofspiel-4, "G4": $k = 4$). Each round a prize card is revealed, in

an order chosen at random; both players simultaneously bid one card from their hand; the higher bid takes the prize. Players observe who won the round but not the opponent's bid card, so the opponent's spent — hence remaining — cards are hidden: the hidden hand is the private state, and the prize sequence plus the per-round win/loss/tie outcomes are the public state. We place the depth-limit cut at the chance boundary where the second prize is about to be revealed — i.e., after the first resolved round. The public information there is which prize was contested and how the bid comparison went, taking $K = 4 \times 3 = 12$ distinct values at G4 (9 at G3, 15 at G5). Each of these K *public keys* roots an identically shaped continuation subgame, because the remaining structure of the game depends only on how many cards remain, not which. Each key's subgame carries the hidden bid pairs consistent with its outcome as private-deal instances; summed over keys these give $B = 64$ *cut instances* at G4 (27 at G3, 125 at G5).

Why naive depth limits are unsound. Truncating the lookahead and valuing cut nodes with belief-blind numbers fails in imperfect-information games: the value of a cut node depends on what both players believe about the hidden information — a poker leaf is worth more when your range is credibly strong — so a leaf must be valued conditional on the belief pair. Value the cut with belief-blind numbers and the solver above the cut learns strategies a real opponent exploits [8]. The sound instantiations — DeepStack's continual re-solving [43], ReBeL [12], Student of Games [49] — train a value network $\hat{V}(\beta)$ over PBSs and re-solve subgames at play time and, critically for this paper, at *train* time; value-function error propagates to exploitability with known bounds [27].

The training loop. The tabula-rasa loop we accelerate is iterated continual resolving with *per-belief finite-budget CFR+ targets*: in each round, (i) sample a belief β at every cut public state; (ii) **run the fixed CFR+ budget on every below-cut subgame at the sampled belief** and read off the normalized continuation values $V_T(\beta)$; (iii) regress the PBS value net on $(\beta, V_T(\beta))$ rows; (iv) periodically re-solve the trunk (the above-cut portion of the game, which the agent solves directly; Figure 1) against the current net leaf and measure exploitability of the assembled policy. Step (ii) is the *inner re-solve* and is where the cost lives. Leaf values use a fixed *normalized* PBS convention,

$$v_i(h_i \mid \beta) \;=\; \frac{\sum_{h_{-i}} r_{-i}(h_{-i})\,\mathrm{EV}_i(h_i, h_{-i})}{\sum_{h_{-i}} r_{-i}(h_{-i})},$$

where h_i ranges over player i's private states consistent with s_{pub}, r_{-i} is the opponent's entry reach vector of β, and $\mathrm{EV}_i(h_i, h_{-i})$ is player i's expected continuation payoff at the cut instance identified by the private pair (h_i, h_{-i}) — under the re-solved subgame's average strategy when generating targets, and under the exact full-game continuation in the round-trip validation. The convention is validated by a round-trip gate: valuing the trunk through this leaf interface reproduces exact full-game above-cut values to $\sim 10^{-15}$ (the gate mechanics and the zero-reach edge case are in Appendix A).

Where the population arises. The inner re-solve is never one subgame: a single belief sample requires solving *all* K keys' subgames at once, and target generation repeats that K-population solve for every sampled belief in every round. This is a representative workload of the method family — DeepStack's value networks were trained on the order of 10^7 independently re-solved subgames [43]. Kim & Sandholm make the mirror observation from the within-tree side: their GPU-CFR framework is particularly useful for "games that are played in a repeated setting, whose individual game trees usually differ by little to none across repetitions" [25] — the population of same-topology subgames is the train-time analogue of that observation, taken across the subgames of one resolving sweep rather than across episodes. One central delimitation for the novelty claim: *within* a key, solving for all private-deal instances simultaneously is the classical range-vector decomposition of CFR [22, 23], used in DeepStack's own resolver and carried identically by *both* of our timing arms; the *across-key*

Algorithm 1 Iterated resolving with population-fused target generation

for training round $r = 1, \ldots, R$ **do**
  sample PBS beliefs $\beta_{j,k}$ for each training belief j and cut public key k
  for belief sample j **do**
    solve all K below-cut subgames at $\{\beta_{j,k}\}_{k=1}^{K}$ for fixed CFR+ budget T ▷ sequential: host loop over k; fused: one population solve
    extract normalized finite-budget targets $V_T(\beta_{j,k})$
  end for
  update value net on the collected $(\beta, V_T(\beta))$ rows
  periodically solve the trunk against the current value net and score exact NashConv
end for

axis K is what this paper adds (full delimitation in §4.5 and §7).

2.3 GPU execution economics

The last prerequisite is the cost structure of GPU execution, for the reader who knows ML but not GPU performance. Every kernel launch costs a fixed amount of host-side overhead — on the order of microseconds — regardless of how much work the kernel does. A workload made of many small kernels is therefore *launch-bound*: the device idles between launches, and throughput is set by the host's ability to feed it, not by arithmetic. This launch-bound regime is, we believe, the same phenomenon Kim & Sandholm [25] report from the other side: their GPU implementation is slower than OpenSpiel's C++ on small games (Kuhn and Leduc poker), where kernel-launch overhead dominates useful work. Throughout, *eager dispatch* means each tensor operation is issued to the GPU as its own kernel launch the moment the Python interpreter reaches it, in contrast to captured or compiled execution, which records a kernel sequence once and replays it. A workload whose individual kernels fill the device is *saturated* (compute-bound) and gains nothing from more concurrency. Batching many same-shaped small problems into one kernel — so the device does the same arithmetic under $O(1)$ launches — is the founding insight of batched BLAS [1, 2], and roofline-style throughput arithmetic [57] frames the cost model we build in §6.1. CUDA Graphs [44] are the rival lever: record a fixed kernel sequence once, then replay it with a single launch — amortizing launch cost without fusing any work (we analyze this alternative for our kernel in §4.6). Finally, define the *fill fraction* $\varphi \in (0, 1]$ informally as the fraction of the device one subgame's kernels occupy: roughly $1/\varphi$ such subgames fit before the device saturates. This is all a reader needs to predict the two-regime model of §6.1: co-solving N subgames should pay about $N\times$ while $N \lesssim 1/\varphi$, and plateau beyond.

Notation and project vocabulary.

Three project terms used from the abstract on: a **gate** is a hard pass/fail check run as part of the experiment pipeline ("verified/gated at x" = certified to x by such a check); the two **arms** of an experiment are the two runs sharing one seed — fused and sequential — in the clinical-trial sense; **equal-band-faster** is this paper's claim form — the same final exploitability band, in less wall-clock.

There are two population dimensions in the implementation. K is the conceptual across-solve axis: the number of public-key subgames one resolving sweep must solve. B is the flattened cut-instance dimension after each key's compatible private-deal rows are expanded. The implementation carries B in tensor axes, while the cost model varies the number of co-solved public-key subgames as N.

Table 3: Notation.

symbol	meaning
K	public keys at the cut — the across-solve fusion axis
B	cut instances: private-deal combinations, summed over keys
N	number of co-solved subgames in controlled sweeps
φ	fill fraction: fraction of the device one subgame's kernels occupy
m	fused marginal-cost probe value (§6.1)
$N^* \approx 1/m$	regime boundary estimated by the probe
f	inner-resolve fraction of the baseline loop's wall-clock
S_{inner}	fused-vs-sequential inner-resolve speedup
T	CFR+ iterations per solve
c_{solve}	wall-clock per solve-iteration
n_I	below-cut infosets (rows of the regret table)
$A_{\max}$	maximum actions per infoset (padded)
$R,\ S,\ \sigma$	cumulative-regret table, strategy-sum table, current strategy

3 The Cost of Iterated Resolving

Before describing the mechanism, we quantify the problem it solves. The three facts below are measured in the experiments of §6 (artifacts in §10); this section states each once, in its motivating role.

Fact 1 — the loop *is* the inner solve. In our instrumented training runs, the inner re-solve (step (ii) of the §2.2 loop) accounts for **98.4%** of baseline loop wall-clock at the Goofspiel-4 operating point, and **86.1%** at Goofspiel-5 (median of 3 quiet-host seeds; §6.2, §6.4). By Amdahl's law [3], accelerating the inner re-solve is, to first order, accelerating training itself.

Fact 2 — the GPU is starving, not slow. At Goofspiel-4 scale, eager per-tree GPU dispatch is **∼4.3× slower** per belief than the same kernel run on the host CPU (GPU-sequential 2.353 s vs CPU-sequential 0.547 s per belief).[1] Small float64 subgames pay for kernel launches, not arithmetic — solved one tree at a time, the GPU loses to its own host. The same artifacts set the paper's cross-substrate floor: against the strongest non-fused arm we measured (the same kernel, sequential, on the host CPU), the best-vs-best Goofspiel-4 win for the fused GPU solver is **∼2.7×** (§4.6).

Fact 3 — fusion removes the per-subgame launch cost. Fused solve time is essentially **flat in the number of co-solved public-key subgames** — 197.6 → 202.4 ms as the Goofspiel-4 sweep size goes 1 → 12 — while sequential per-tree time grows linearly (§6.1). Added subgames ride along nearly free until the device fills.

The lever taxonomy, compressed. Prior work claims every factor of Eq. (1) except the one Facts 2 and 3 point at. N_{solves} is reduced by TurboReBeL's single-sample multi-iteration data generation [35]; T by regret-minimizer design — DCFR/PCFR+ and the meta-learned predictive minimizers, deployed on each subgame sequentially [9, 18, 52, 53]; sampling variance by the MCCFR family [31, 48, 40, 51]; subgame size by learned abstractions [29]. The remaining factor, c_{solve}, has been attacked only *within* a single tree, by the GPU-CFR line [24, 25]. The cost-per-solve-across-the-population axis — making each CFR+ iteration cheaper by amortizing it over many subgames at once, exactly — is not isolated in prior published work as an end-to-end population-level lever; §7 gives the lever-by-lever discussion and the related-work delimitation.

[1] The CPU cells are single determinism-gate runs without warmup/repeat discipline, so the cross-substrate readings are provisional on matched full-configuration CPU timings; the full 2×2, its provenance, and its caveats are in §4.6 and Appendix C.

Algorithm 2 Per-tree CFR+ (the sequential arm)

compile each key k's topology once (shared shape, per-key data)
for $t = 1, \ldots, T$ **do** ▷ one CFR+ iteration
  for $k = 1, \ldots, K$ **do** ▷ **host loop: one launch sequence per tree**
    regret-match: $\sigma_k \leftarrow$ clamp-normalize(R_k)
    forward pass: reaches root $\rightarrow$ leaves on tree k
    backward pass: counterfactual values leaves $\rightarrow$ root; deposit regret contributions into R_k
    CFR+ update: clamp R_k at 0; alternate player; update S_k
  end for
end for

Algorithm 3 Fused population CFR+ (this work)

compile the shared topology once (population data: length-B arrays)
for $t = 1, \ldots, T$ **do** ▷ one CFR+ iteration: **one launch sequence; the population is a tensor dimension**
  regret-match: $\sigma \leftarrow$ clamp-normalize(R) ▷ all legal rows at once; pad slots masked
  forward pass: batched gather $\times$ multiply over level groups ▷ $[G, B]$ blocks
  backward pass: batched contractions + scatter-add ▷ $[G, A, B, 2]$ by per-instance infoset id
  CFR+ update: clamp at 0; alternate player; update S ▷ masked average-strategy normalization
end for

4 Method: Population Fusion

4.1 Mechanism overview

The whole mechanism is three sentences. (1) All K cut subgames share one topology, differing only in data — entry reaches, payoffs, infoset identities (§2.2). (2) We compile that topology *once* into a flat structure-of-arrays layout whose nodes are grouped by level and shape (§4.2). (3) Each CFR+ iteration of *all* subgames then executes as one fixed sequence of batched tensor ops (§4.3): the population axis moves from a host-side loop — one kernel-launch sequence per subgame — into the innermost tensor dimension of every op, so the per-iteration launch count is $O(\#\text{level-shape groups})$, independent of the co-solved public-key count. Figure 2 walks the mechanism through a tiny two-subgame example, and Algorithms 2 and 3 state the two timing arms side by side: identical four steps, identical kernels, differing in exactly one line. (Shape annotations in Algorithm 3: G is the node count of the level-shape group an op ranges over, A the group's per-infoset action count, and B the flattened cut-instance dimension of §2.2; full notation in §4.3.)

The two algorithms run the same kernels on the same data; they differ in exactly one visible line — the host loop over keys versus the population tensor dimension. This is also the baseline-fairness claim of §4.6 in pictorial form: the sequential arm is not a weak baseline but the identical kernel at a different batching granularity.

4.2 Topology compilation

Our subgame-construction layer (`DepthLimitedGame`) enumerates an OpenSpiel game [33] once into a tagged tree of terminal, chance, decision, and **cut** nodes; the cut predicate is the one game-specific

hook, and each cut node records its public key k, each player's private index (i_0, i_1) at that key, and the continuation subtree. Compilation then proceeds in four steps. **(1)** Tag the shared below-cut topology from the cut nodes (the cut sits at a public chance boundary, so instances differ only in private deals — hence in entry reaches, payoffs, and infoset identities, not in shape). **(2)** One recursive descent over the *lockstep population* — all B instances advanced through the shared topology together — emits one record per topology node whose per-instance data are length-B arrays: terminal payoffs $[B, 2]$, chance branch probabilities $[B, n_{\text{branch}}]$, and per-instance global infoset ids $[B]$ at decision nodes. **(3)** Group the records **by level and shape**: forward groups collect all level-L children by parent kind (decider 0, decider 1, chance); backward groups collect all level-L non-terminals by $(\text{player}, n_{\text{actions}})$ or $(\text{chance}, n_{\text{branch}})$. **(4)** Absorb variable depth, heterogeneous branching factors, and interleaved chance nodes into these groups plus padded legal-action masks over the union of below-cut infosets. A reimplementer must get three things right: the global infoset-id space spans the whole population (it indexes the rows of R); the ids are *per-instance*, so the same topology node maps to different rows in different instances; and pad slots must be masked out of every subsequent update.

4.3 The fused CFR+ iteration

What one CFR+ iteration computes, in the vocabulary of §2.1: step 1 turns accumulated regrets into the *current* strategy — the clamped normalization of regret matching. Step 2 pushes *reach probabilities* from the entry reaches down to every node — "how likely is play to get here, and who contributed". Step 3 pulls *expected values* from the terminals back up, forming counterfactual action values weighted by opponent-and-chance reach, and deposits each infoset's regret contributions. Step 4 is the CFR+ update: clamp cumulative regrets at zero, alternate the updating player, and accumulate the own-reach-weighted average strategy — the quantity whose time average converges to equilibrium. Fused, each step becomes one batched operation per level-shape group.

Notation: B cut instances and K public keys as in §2.2; n_I below-cut infosets in the population union with at most $A_{\max}$ actions each; $R, S \in \mathbb{R}^{n_I \times A_{\max}}$ the global regret and strategy-sum tables; σ the current strategy from regret matching; $r^{\text{own}}, r_{-i}, r^c$ own, opponent, and chance reaches; G as defined in the Algorithm 3 gloss (§4.1). Each CFR+ iteration is a fixed sequence of batched tensor ops with *no per-iteration Python tree recursion*:

1. *Regret matching* [19]*:* one clamped-normalize over the global regret table R (float64), $\sigma(I, a) = [R(I, a)]_+ / \sum_{a'} [R(I, a')]_+$ over legal actions, uniform over legal actions where the denominator vanishes (illegal slots are mask-padded out of every update).

2. *Forward reach pass:* for each forward group, child reaches are one gather + elementwise multiply — $r^{\text{own}}_{\text{child}} = r^{\text{own}}_{\text{parent}} \odot \sigma[\text{id}, \text{slot}]$ for decision parents, where $\sigma[\text{id}, \text{slot}]$ is the parent infoset's current-strategy entry for the action leading to the child, and $r^c_{\text{child}} = r^c_{\text{parent}} \odot p$ for chance parents, where p is the branch probability — over $[G, B]$ blocks.

3. *Backward value pass:* for each backward group, child EVs are gathered as $[G, A, B, 2]$ tensors, where A is the group's per-infoset action (or chance-branch) count, and contracted against σ (or chance probabilities); counterfactual action values $r_{-i}\, r^c$ EV (opponent reach times chance reach — the counterfactual reach) are scattered into the global accumulator with `index_add_` over the per-instance infoset ids.

4. *CFR+ update:* alternating-player regret update with clamp at zero, own-reach-weighted strategy sums, linear averaging (on the interaction of alternating updates with averaging in CFR+, see [14]).

Table 4: Per-CFR-iteration operation structure. L is the number of level-shape groups, K the public-key population size, and B the flattened cut-instance tensor dimension. Arithmetic work scales with the population in both views; fusion removes the host-side serial launch sequence over keys.

Quantity	per-tree sequential arm	fused population arm
Host launch sequences per CFR+ iteration	KL	L
Tensor arithmetic work	$\Theta(B \cdot \text{group work})$ across K calls	$\Theta(B \cdot \text{group work})$ in one batched call
Regret / strategy rows	disjoint per key	union table $R, S \in \mathbb{R}^{n_I \times A_{\max}}$
Serial host loop over public keys	yes	no
Correctness condition	per-key CFR+	same update plus disjoint below-cut rows

Per-iteration kernel-launch and op count is $O(\#\text{level-shape groups})$ — fixed by the topology and *independent of the co-solved public-key count*, which appears only as a tensor dimension (the arithmetic work per op scales with it, which is precisely what fills the device). In the work–depth vocabulary of [25], the population enters the *work*, not the *depth*: per-iteration work scales linearly with the cut-instance tensor dimension, while the launch/op count — the host-side serial term — stays $O(\#\text{level-shape groups})$, independent of it. (We use "GEMM-shaped" — GEMM: general matrix–matrix multiplication — loosely, here and in the title: the per-iteration ops are gathers, scatters, and elementwise products plus small dense contractions, executed as batched tensor kernels.) Running example, one G4 resolve sweep: $K = 12$ keys, $B = 64$ instances, 300 CFR+ iterations — the sequential arm issues 12 launch sequences per iteration $\times$ 300 iterations; the fused arm issues one per iteration.

4.4 Reuse across resolves, and the value pass

Between solves only the entry reaches change: the two $[B]$ entry-reach vectors are rewritten and the compiled topology is reused for the whole training run. Compilation is one-time infrastructure and is excluded from all solve timings symmetrically in both arms — conservatively for the fused claim, since the sequential arm requires K per-key compiles plus the shared one (the corresponding setup costs are: topology compile 0.09–9.4 s per game, `torch.compile` $\leq$9.3 s, CUDA-Graph capture $\leq$0.15 s — all excluded symmetrically). After the final iteration, a value-only backward pass under the average strategy returns the per-instance continuation EV $\text{cont} \in \mathbb{R}^{B \times 2}$, from which normalized per-private leaf values (§2.2) are reconstructed in $O(B)$ — so target generation and trunk leaf queries also avoid any Python subtree walk.

4.5 Parity: the fused solve matches finite-iteration CFR+ outputs

We require the fused solve to return the same finite-iteration CFR+ average strategies and leaf values as the per-tree solve — exact in the verified sense defined by the gates of §5. Three clarifications delimit what is and is not claimed. First, *the within-key coupling is classical*: within one public key, multiple cut instances write reach-weighted counterfactual values into the same infoset rows — this is range-vector CFR [22, 23], present identically in both arms, and not what fusion adds. Second, *across* keys the below-cut infoset rows are disjoint (post-cut infosets condition on the public

outcome), so across-key fusion changes only floating-point accumulation order and op scheduling — which is exactly why parity is mathematically expected:

Definition (well-formed compiled population). A fused population is well formed when the game has perfect recall; the cut is public; below-cut infoset-row sets $\mathcal{I}_k$ are pairwise disjoint across public keys k; each fused instance uses the same action ordering, legal-action masks, chance/terminal/payoff tensors, and per-instance infoset-id map as its per-key solo solve; and both arms start from identical R, S tables and run the same finite CFR+ budget T with the same masked regret matching, alternating-player updates, clamp, and averaging rules.

Proposition 1 (finite-iteration parity of fused CFR+). For any well-formed compiled population, running (a) the fused solver on all K keys and (b) the same CFR+ kernel on each key alone gives, in exact arithmetic, identical iterates on each key's rows and instances at every iteration. In particular both arms return identical finite-budget average strategies and leaf values. This is an output-parity statement for a fixed update budget, not a convergence-rate, lower-exploitability, or stronger-equilibrium claim. *(Proof sketch: Appendix A.)*

The observed parity matches: maximum average-strategy difference 0 at Goofspiel-3 and Goofspiel-4 and 4×10^{-6} at Goofspiel-5 (float64; range 0–7×10^{-6} across all parity-tested games, including Leduc poker and liar's dice), with a CPU unit gate at $< 10^{-9}$ (§5). Third, the *genuine* engineering nontriviality is elsewhere: recovering these semantics through a level-grouped compilation of irregular structure (variable-depth branches, mixed chance/decision levels, per-infoset action counts, mask padding); float64 discipline through hundreds of compounding clamped fixed-point iterations with linear averaging (a float32 ablation drifted by ~ 0.75 L1 in average strategy at repeated imperfect-information infosets); and integration into a live training loop with topology reuse and $O(B)$ leaf reconstruction. Parity is a *gated property*, not a by-construction guarantee: CFR+ amplifies ordering and padding discrepancies rather than averaging them out, so the gates of §5 are part of the method.

4.6 Baseline: the same kernel, one tree at a time

The baseline arm (**sequential**) is the *same* flat-SoA CFR+ kernel applied to one subgame tree at a time, each with its own precompiled, reused topology — i.e., within-tree GPU-CFR in the sense of [24]: every individual subgame is solved on-GPU with full within-tree parallelism (including the within-key range dimension), but the population is traversed by a host loop with one launch sequence per tree. The two arms execute literally the same code path and differ only in batching granularity (guarded by an assertion in the run harness). The comparison is thus same-device, same-algorithm, same-precision, and matched-seed — meeting the fairness standard Kim & Sandholm [25] themselves apply, down to the double precision they choose to match OpenSpiel. We explicitly *reject* the Python reference tree-walk solver as a timing baseline: it is the correctness reference, but timing against it would inflate the speedup with interpreter overhead and make the comparison a weak baseline. Any reported fused-vs-sequential speedup is thus attributable to population fusion alone, not to GPU-vs-CPU or compiled-vs-interpreted effects.

What the baseline is not. Two stronger non-fused baselines exist and bound the claim; we present both because they are natural comparisons.

(i) The host CPU. Table 5 gives the per-belief inner-resolve 2×2 at Goofspiel-4.

Three readings. **At G4 scale the eager-dispatch GPU-sequential baseline is ∼4.3× slower per belief than the same kernel on the host CPU**; fusion on CPU is worth 1.30×; and the best-fused-vs-best-non-fused comparison (GPU-fused vs CPU-sequential) is ∼2.7× — the practitioner-facing cross-substrate number at G4, provisional on matched budgets. The reading

Table 5: Per-belief inner-resolve cost at Goofspiel-4 (seconds per belief; lower is better). Sources and caveats: the CPU cells are single determinism-gate runs without warmup/repeat discipline; the GPU cells are seed-0 full-configuration end-to-end runs; matched full-configuration CPU timings remain outside this version. Protocol details: Appendix C.

	sequential (per-tree)	fused (population)
host CPU	0.547	0.42
GPU (eager dispatch)	2.353	0.201

we draw is the regime model's own: in the launch-bound regime — tiny float64 subgames — eager per-tree GPU dispatch lost even to the host CPU, and *in these runs fusion is what made the GPU the faster substrate at this scale.* At G5 no CPU timings exist; with 236,450 infosets the arithmetic-dominated comparison is expected to invert, but that is untested and we do not claim it.

(ii) A launch-amortized sequential arm (CUDA Graphs / `torch.compile`) — now measured. The launch overhead that fusion amortizes is also the target of CUDA-Graphs capture and `torch.compile(mode="reduce-overhead")`. We first checked the kernel for CUDA-Graph capturability: every tensor shape in the iteration body is static and the loop has no synchronizing host reads, so there is **no structural barrier to capture** — the obstacles are engineering-level (two iteration-indexed Python scalars and out-of-place state rebinding, both with standard remedies; the full capturability analysis, and the confirmation that those remedies are exactly what the implementation needed, are in Appendix C). Our prediction, stated for the record before measuring: capture can recover the launch-gap component but not the short-kernel occupancy component, so it should close part — not all — of the launch-bound gap. The ablation is now measured, in both directions — a six-arm matrix {fused, sequential} × {eager, compiled, graph-captured}, parity-gated, cross-game, with sweeps (§6.5). The outcome in one sentence: the prediction holds for pure capture (graphs close 73–80% of the per-tree launch gap at G3/G4 but the amortized sequential arm remains linear in the number of co-solved public-key subgames), `torch.compile`'s kernel re-codegen additionally reaches eager-fused parity at G5, and under tool symmetry — the same tooling applied to the fused arm — fusion retains speedup $\approx N$ launch-bound and $\sim$1.9$\times$ marginal at saturation. The launch-bound speedups of §6.1–§6.4 are measured against an eager-dispatch single-stream sequential arm; §6.5 provides the tool-symmetric restatement.

A natural objection remains: *is this just batching embarrassingly parallel work?* The short answer is that no single ingredient is claimed novel in isolation — the claim is the conjunction of the structural identification, the parity-gated fused solver, the predictive regime model, and the Amdahl-surviving end-to-end win; §8 states the objection and the full answer.

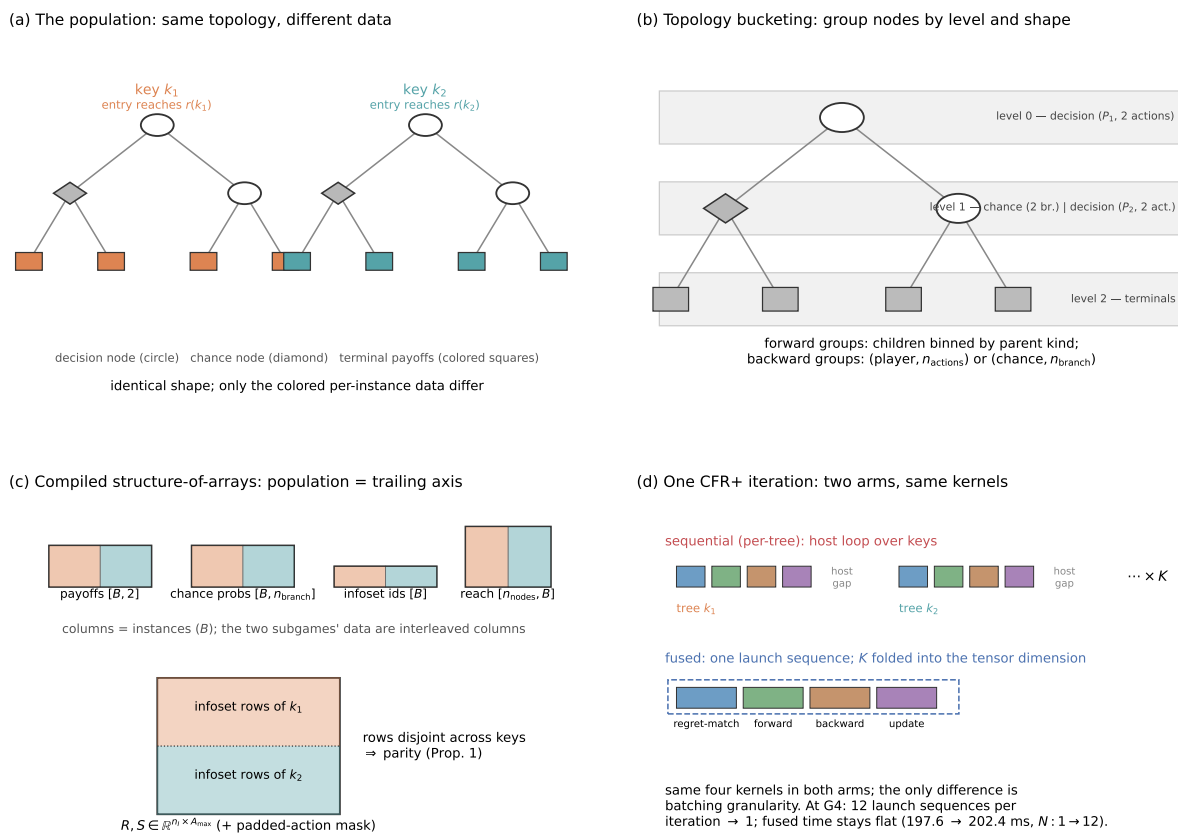

Figure 2: Population fusion on a tiny two-subgame example. *(a)* Two same-topology depth-limited subgames rooted at public keys k_1, k_2 on the trunk's cut frontier; per-instance data (entry reaches, terminal payoffs) shown in color — same shape, different data. *(b)* The same topology drawn once, with nodes binned into level-shape groups: forward groups keyed by parent kind, backward groups keyed by (player, n_{actions}) or (chance, n_{branch}). *(c)* The compiled structure-of-arrays: per-node data become length-B arrays with the population as the trailing axis, and the global regret/strategy-sum tables $R, S \in \mathbb{R}^{n_I \times A_{\max}}$ hold the two keys' infoset rows as disjoint blocks — visually pre-stating the parity argument of §4.5. *(d)* One CFR+ iteration as kernel timelines: the sequential arm issues one launch sequence per tree with host gaps between them; the fused arm issues one launch sequence total, with K folded into the tensor dimension. At G4 this is 12 launch sequences per iteration → 1, and fused time stays flat as the population grows (197.6 → 202.4 ms for N: 1→12; §6.1).

5 Experimental Setup

Games. The Goofspiel ladder of §2.2 [45] with 3, 4, and 5 cards (G3/G4/G5), loaded from OpenSpiel (simultaneous-move game through the turn-based wrapper; imperfect information, random prize order). The depth-limit cut is the prize-reveal chance boundary after the first resolved round (a single cut level; scope in §8). The ladder spans:

game	public keys K (fusion axis)	cut instances B	infosets
G3	9	27	90
G4	12	64	3,608
G5	15	125	236,450

At G5 the compiled topology has 26,773 nodes over 10 level groups and the fused solve runs at ≈6.2 ms per CFR+ iteration in-loop, including leaf reconstruction — indistinguishable from the pure-solve microbenchmark figure (≈6.2 ms/iter). The implementation is game-agnostic in interface (the cut predicate and public-key function are the only game hooks; correctness is parity-tested on Leduc poker and liar's-dice cut structures), but performance generality is demonstrated only on Goofspiel: all end-to-end timing claims in this paper are on this ladder.

Metric. Exploitability is *exact* NashConv [32], $\sum_i \left(\max_{\sigma'_i} u_i(\sigma'_i, \sigma_{-i}) - u_i(\sigma)\right)$, where u_i is player i's expected utility under the assembled profile σ (σ here denotes the assembled full profile, distinct from the within-solve current strategy of §4.3) — in plain terms, how much an optimal opponent could gain in total: 0 is exact Nash, and uniform random play scores 1.4167 on G4. It is computed by OpenSpiel's exact best response on the assembled full policy: the trunk average strategy spliced onto the below-cut continuation re-solved at the on-policy belief. We additionally report the DeepStack-style safe-resolving gadget continuation [13, 43] — a standard construction that re-solves a subgame so the result is guaranteed no worse against any opponent strategy; since gadget equilibria are refinement-sensitive [30], band claims use the on-policy re-solve continuation, with gadget numbers alongside. No sampled or approximate exploitability is used anywhere.

Protocol. One configuration, fixed in advance of the runs, with no sweeps (verifiable from the released commit history): 8 target rounds × 40 sampled beliefs per round; 300 CFR+ iterations per inner re-solve; beliefs drawn per key as symmetric Dir(α) over each player's private states, with α resampled uniformly at random from $\{0.3, 1.0, 3.0\}$ independently per player, per key, per sampled belief; trunk CFR+ 200 iterations; PBS value net = MLP (hidden 128), 400 epochs per round, buffer cap 8,000; exact measurement every 2 rounds and at the end (on-policy continuation re-solved with 600 iterations; gadget 1,500).

Seeds and pairing. Each end-to-end experiment runs the §2.2 loop twice per seed — the two *arms*, fused and sequential, sharing the seed and hence the identical belief sequence and (up to CUDA atomics, below) identical targets and trajectories. G4 uses 5 paired seeds. At G5 — the saturated regime of §6.1 — the same paired-arm design runs a shortened budget (4 target rounds × 40 beliefs; exact measurement at the final round only; the safe-resolving gadget evaluation skipped), all other settings unchanged. The headline G5 numbers come from a quiet-host re-run of this design at 3 paired seeds (seeds 0–2; machine otherwise idle); the original 2-seed pair, whose sequential seed-1 arm was host-load-contaminated, is superseded and retained as the motivating diagnostic (§6.4; forensics in Appendix B.3). The shortening was a wall-clock budget decision taken before that contamination was discovered, not a protocol response to any result; a full-protocol (8-round) G5 paired run at $n \geq 5$ with a pre-registered band margin — needed for a CUDA non-divergence test, not for the timing claim — is planned. The 5-seed G5 exploitability band of §6.4 uses the full 8-round configuration (seeds 0–4, fused path).

Aggregation. Every fused-vs-sequential end-to-end speedup in §6 is the median of per-seed ratios; the §6.1 microbenchmark speedups are ratios of median-of-7-repeat timings (no seed axis); and the secondary time-to-band speedups are ratios of the per-arm median reach times. Where $n \leq 2$ we report the per-seed values themselves and do not dress the midpoint as a robust statistic. Rounding conventions and the rationale for each rule are in Appendix B.1.

Descriptive band check. Two arms pass the descriptive band check when each arm's median final NashConv lies within the other arm's per-seed 10th/90th interval — a deliberately descriptive criterion that is weak at n=5: it certifies the absence of gross divergence, not statistical equivalence (formal definition and full self-critique in Appendix B.2).

Timing discipline. GPU sections are timed with `torch.cuda.Event` pairs bracketed by `torch.cuda.synchronize()`, with wall-clock recorded alongside. One sharp caveat we state as a finding (encountered in our G5 runs): **GPU-event spans that bracket a host-driven launch loop are host-load-sensitive** — the event pair measures stream-timeline elapsed time *including* gaps where the device waits for host submission, so under host CPU contention a launch-heavy (sequential) arm's event spans inflate while a fused arm, issuing far fewer launch sequences, stays fed. (Consistent with this, the recorded GPU-event totals equal the wall-clock totals to the millisecond in these runs — the events bracket wall-equivalent regions; forensics in Appendix B.3.) The cost-per-solve microbenchmark uses 3 discarded warmup runs and the median of 7 repeats per point, on precompiled topologies (pure solve time). In the training loop, per-phase timers separate inner-resolve / net-train / measurement; the timed inner-resolve region covers exactly the per-belief population solve plus leaf reconstruction, while host-side row assembly (identical work in both arms) and one-time compilation are excluded. Measurement rounds run through the fused path in *both* arms — the evaluation protocol is fixed and identical — and their cost is tracked separately from the training-loop wall-clock comparison. **Total wall-clock (`total_wall`)** is the wall-clock of the training loop — inner re-solve + net train + measurement phases, identically bounded in both arms; it excludes game enumeration, one-time topology compilation, and the post-loop gadget pass (all symmetric or conservative for the fused claim; §4.4, Appendix D.4).

Exactness gates. Two hard gates certify that the two timing arms are the same algorithm. (1) A CPU exact-parity unit test (`test_fused_vs_sequential_soa_parity`): fused-over-all-keys vs sequential-per-key average strategies *and* leaf values agree to $< 10^{-9}$. (2) A CPU determinism run of the training loop in both modes — the full loop structure at a reduced budget (4 rounds $\times$ 10 beliefs; disclosed here and in Appendix F): the trajectories (validation MAE per round, measured exploitability) are **identical to all recorded digits** (the recorded values and threshold fine print are in Appendix B.2). This gate discipline has independent motivation: Kim & Sandholm [25, App. D] document that CFR iterates diverge across implementations by up to an order of magnitude in peak exploitability — our gates exist precisely to rule this class of divergence out of the comparison. On CUDA, however, `index_add_` uses atomics, so floating-point accumulation order is non-deterministic; across 8 rounds $\times$ 40 beliefs $\times$ 300 iterations these perturbations *compound through training*, producing per-seed fused-vs-sequential final-NashConv differences of median 0.0144 (max 0.0389) at G4. The CPU determinism gate proves algorithmic equivalence, not unbiased CUDA drift. On CUDA we have no evidence of systematic bias: the G4 seed signs (fused higher on 2/5, lower on 3/5) are descriptive and underpowered — **no detectable bias (n=5; a sign test at this n cannot exclude moderate bias)** — and pooling the quiet-host G5 pairs gives fused-higher on 4/8 (§6.4). We therefore gate algorithmic equivalence on CPU, where it is exact, and use CUDA runs for timing and *distributional* (band-level) comparison only. We flag this compounding-atomics effect as a finding in its own right (contribution 5): naive per-seed equivalence checks of batched GPU CFR through a training loop will spuriously fail.

Search-free boundary protocol. As a per-compute sanity boundary we run NFSP [20]

— the canonical search-free near-Nash learner — on the same game object, scored by the same exact NashConv, under one published OpenSpiel configuration (not swept, not Goofspiel-tuned), evaluated in average-policy mode, with a pre-registered replacement criterion recorded in the artifact. NFSP executes on the host CPU (the OpenSpiel PyTorch implementation's default device, which our run does not override) while both resolving arms use the GPU, so the §6.3 wall-clock comparison crosses platforms — one more reason it is directional only. Current-policy gradient baselines (RPG/QPG [50]) are rejected for the methodological reason given in Appendix D.2, and the strongest search-free line we are aware of (VRPO [17]) is a cited boundary, not reimplemented (rationale also in Appendix D.2) — the primary baseline in every experiment remains the sequential arm itself: the same kernel, with gated parity.

Hardware. All timings: one consumer GPU (NVIDIA RTX 3070 Ti, 8 GB; FP64 throughput 1:64 of FP32), PyTorch float64 tensor ops; exactness gates run the identical code on CPU. The mechanism and the regime model of §6 are stated hardware-independently; the model's *parameters* — launch overhead, device capacity, and hence the regime boundary — are device-specific (§8 discusses the FP64 asymmetry's direction), and a multi-device replication is deferred to future work.

6 Results

We present five result blocks, in the order of the argument: the cost-per-solve microbenchmark and the two-regime model it induces (§6.1); the end-to-end training win at Goofspiel-4, where the model's launch-bound prediction is tested against a real training loop (§6.2); a directional search-free calibration (§6.3); the Goofspiel-5 scale results — the 5-seed exploitability band that the fused solver makes affordable, and the paired end-to-end point at the *saturated* end of the regime model (§6.4); and the launch-amortization ablation, which re-runs the cost-per-solve comparison with CUDA-Graphs/`torch.compile` tooling on each arm — first asymmetric, then tool-symmetric, followed by a G4 tool-symmetric end-to-end check (§6.5). Every speedup follows the §5 aggregation convention; every exploitability number is exact NashConv. The G4 training claims are equal-band-faster on CUDA; the G5 paired run is timing-transfer evidence in the saturated regime, with CPU/single-solve parity and a pending full-protocol CUDA non-divergence test. The one comparison of a different kind — the 2.07× target-construction result in §6.4 — is delimited there as such.

6.1 Cost-per-solve: microbenchmark and the two-regime model

We first measure the cost-per-solve lever in isolation: solve the full cut population of each Goofspiel-ladder game once with the fused solver (all K keys in one batched solve) and once with the *same* SoA kernel applied per key — one subgame tree per public key, i.e., the per-tree within-tree GPU-CFR baseline of §4.6. Timings are GPU-event medians of 7 repeats after 3 discarded warmups, 300 CFR+ iterations, on precompiled topologies (K, B, infosets per game in the §5 ladder table).

Finite-iteration output parity is exact in the gated sense: maximum average-strategy difference between the fused and sequential solves is 0 at G3/G4 and 4×10^{-6} at G5 (float64; the range across all parity-tested games, including Leduc and liar's dice, is 0–7×10^{-6}). The speedup costs nothing in the measured solver outputs.

The non-monotone speedup column ($8.77 \to 11.69 \to 2.37$) is not noise — it is the regime structure, and the median speedups are ceilinged by K, the across-key fusion axis (per-seed values can exceed the ceiling slightly through measurement noise and asymmetric launch overhead; §6.2). A controlled sweep holding per-subgame size fixed and varying the number N of co-solved subgames (keys) — the *key sweep*, Figure 3 — separates the two regimes. On G4, whose individual subgames under-fill the device, fused time is essentially flat — 197.6, 199.5, 200.1, 201.4, 202.4 ms

Table 6: Cost-per-solve bake-off (full cut population, 300 CFR+ iterations).

game	fused (ms)	sequential (ms)	speedup
G3 (K=9)	119.2	1,045.6	**8.77**×
G4 (K=12)	203.1	2,374.3	**11.69**×
G5 (K=15)	1,866.7	4,427.4	**2.37**×

at $N = 1, 2, 4, 8, 12$ — while sequential time grows linearly, so the speedup tracks N nearly exactly: 1.0/2.0/3.96/7.9/11.7× (the key sweep is a separate run from the microbenchmark table row; their values agree within 0.4%). This is the *launch-bound* regime: the device is paying per-launch overhead, not arithmetic, and fusion amortizes the launches (against an eager-dispatch sequential arm; §4.6 — §6.5 re-tests the comparison with launch-amortized tooling on both arms). On G5, whose deeper trees individually saturate the device, fused time grows with N (291 → 1,870 ms at $N = 1 \to 15$) and the speedup saturates: 1.0/1.59/2.08/2.24/2.39× at $N = 1, 2, 4, 8, 15$ — residual launch-overhead recovery on top of an already-busy GPU (the sweep endpoint 2.39× and the microbenchmark table row 2.37× are separate runs agreeing within 1%; the table row is the value used as the §6.4 prediction). The data are summarized by a floor/slope model:

$$C_{\text{seq}}(N) = a_s + b_s(N-1), \qquad C_{\text{fus}}(N) = a_f + b_f(N-1), \qquad S(N) = \frac{C_{\text{seq}}(N)}{C_{\text{fus}}(N)}. \tag{2}$$

Under the same launch tooling, $a_s \approx a_f$; fusion's measurable advantage is the change in marginal cost, $b_f \ll b_s$. In the launch-bound regime, $b_f \approx 0$, so $S(N)$ tracks N until the device fills. In the saturated regime, $S(N)$ contracts toward b_s/b_f; at G5 under `torch.compile`, §6.5 measures this marginal slope ratio as $129.6/68.0 \approx 1.9$. The simplified $\min(N, 1/\varphi)$ form is therefore the zero-overhead launch-bound envelope and regime intuition, not the formal plateau predictor. Here φ is the *fill fraction* — the fraction of the device one subgame's kernels occupy (we avoid CUDA's narrower warps-per-SM sense of "occupancy"). The envelope is useful for identifying the boundary $N^* \approx 1/\varphi$; the saturated plateau is empirical because residual launch recovery, kernel re-codegen, memory traffic, and occupancy effects remain after the device is already busy. The model is additionally subject to memory capacity: the fused population's state must fit on the device, so the attainable N is capped by a device-specific maximum population $B_{\max}$ — every population in this paper fits on the 8 GB device (up to K=15, B=125), and peak-VRAM-per-arm instrumentation is a deferred reporting item (§8).

How to classify your workload (the regime probe). The regime — though not the exact plateau height — is classifiable from one additional cheap measurement after the N=1 baseline: the **fused marginal-cost probe**. Solve the population fused at N=1 and at a small N>1 and compute the normalized marginal cost $m = \big(\text{fused}(N) - \text{fused}(1)\big)/\big((N-1)\cdot\text{fused}(1)\big)$. Under the affine extrapolation $\text{fused}(N) \approx \text{fused}(1)\cdot\big(1 + (N-1)\,m\big)$, the probe estimates the cost model's own regime boundary: $N^* \approx 1/m$ plays the role of $1/\varphi$, and the classification compares the population size N to be fused against $1/m$. A workload is **launch-bound when** $N \ll 1/m$ — added subgames ride along nearly free and speedup tracks N — and **saturated when** $N \gtrsim 1/m$ — fused time has grown by a multiple of the single-population solve and speedup plateaus at residual launch recovery. On the released data the probe's classifications at the populations actually fused match the sweep-curve labels for all four sweep-covered workloads on this device (probe step = the smallest sweep point above N=1: N=2 for G4 and G5, N=4 for the heads-up

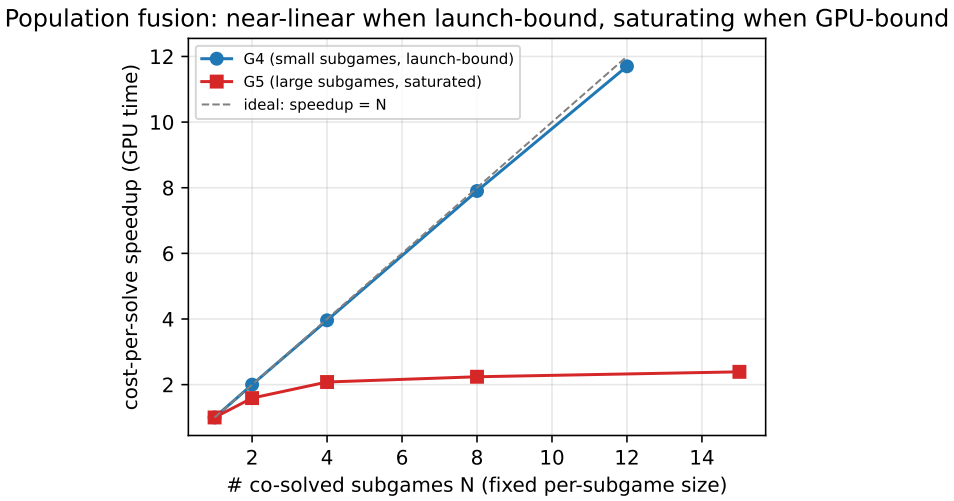

Figure 3: The two-regime cost model in one picture. Cost-per-solve speedup (GPU-event time, fused vs sequential per-tree GPU-CFR; the sequential arm is bracketed end-to-end around its per-tree host loop) as a function of the number N of co-solved subgames (public keys) at fixed per-subgame size. Goofspiel-4 (G4; small subgames, blue) is launch-bound: fused time is ∼constant (197.6 → 202.4 ms for N: 1→12), so speedup tracks the ideal speedup = N line (dashed). Goofspiel-5 (G5; large subgames, red) saturates the device at N=1: speedup plateaus at ∼2.4×. *How to read the two regimes:* on the blue curve every added subgame rides along nearly free — the device was starving — so the curve hugs the diagonal until the device fills; on the red curve one subgame already fills the device, so co-scheduling buys nothing and the residual plateau is launch-overhead recovery only. Timings are medians of 7 repeats after 3 discarded warmups, with 300 CFR+ iterations.

no-limit Texas hold'em (HUNL) classes): G4 $m = 0.010$ (launch-bound; measured in-loop 11.79×, §6.2); G5 $m = 0.277$ (saturated; plateau ∼2.4×, §6.4); the two HUNL census classes are reported with the census itself in §8. Since the labels are read off the same sweep curves, this is an operational definition illustrated on released data; the probe's *predictive* content is carried by the microbenchmark-to-loop transfers of §6.2 and §6.4.

The model makes two predictions tested in the rest of this section: at G4 the in-loop inner-resolve speedup should be ∼11.7× (the N=12 sweep point; §6.2), and at G5 it should be ∼2.4× (the saturated plateau, microbenchmark value 2.37×; §6.4). Train-time iterated resolving structurally generates the launch-bound regime — many small same-topology PBS subgames per iteration — and the abstraction-learning line [29] pushes workloads further into it.

6.2 End-to-end training win, Goofspiel-4

Microbenchmark speedups routinely die inside real training loops; this one survives, because the loop is dominated by exactly the phase we fused. Running the full §2.2 loop under the single advance-fixed configuration of §5, with both arms sharing each of 5 seeds:

- **Inner-resolve speedup 11.79×** (median of per-seed ratios; per-seed range 11.68–12.23) — against the regime model's launch-bound prediction of ∼11.7× from the §6.1 sweep, made before these runs.

- **Total wall-clock speedup 9.99×** (per-seed range 9.97–10.41): median total wall **fused 76.8 s vs sequential 767.2 s** per arm.
- **The Amdahl arithmetic closes ex post.** Write each arm's loop total as inner plus non-inner time, $T = I + O$, and define per-seed $f = I_{\text{seq}}/T_{\text{seq}}$ (the inner-resolve fraction) and $S_{\text{inner}} = I_{\text{seq}}/I_{\text{fus}}$. Exactly,

$$\frac{T_{\text{seq}}}{T_{\text{fus}}} = \frac{1}{(1-f) + f/S_{\text{inner}} + \Delta O/T_{\text{seq}}}, \qquad \Delta O = O_{\text{fus}} - O_{\text{seq}},$$

  and since the non-inner phases perform identical work in both arms (§5), ΔO can only be timing noise, leaving the Amdahl ceiling $1/((1-f) + f/S_{\text{inner}})$ [3]. The inner re-solve is **98.44%** of the sequential baseline loop (98.4% rounded elsewhere); given the measured inner speedup ($S_{\text{inner}} = 11.79\times$; the ex-ante prediction was 11.7×), the implied ceiling is **10.09×** (computed from the unrounded artifact value $f = 0.9844$; this is the *median*-implied ceiling — each seed's ratio is bounded by its own seed-level ceiling, which varies with that seed's f and S_{inner}, and all five seeds satisfy theirs in the artifacts; a seed's total-wall ratio can exceed its own implied ceiling only through noise-scale non-inner timing asymmetry, $\Delta O < 0$) — and the measured 9.99× sits at 99% of it. To be precise about what is prediction and what is accounting: the *microbenchmark predicted the inner-resolve ratio ex ante* (the microbenchmark runs predate the loop runs; the ordering is verifiable from the released commit history, Appendix F); the Amdahl decomposition uses f measured from the loop itself and is an accounting-consistency closure — the residual 1% reflects the non-inner phases together with the across-seed median aggregation. The same decomposition, run in reverse, is the contamination diagnostic of §6.4.
- **Descriptive band check.** This is not an improvement claim. Fused final NashConv median **0.0402** [10th/90th: 0.0238, 0.0533]; sequential median **0.0469** [10th/90th: 0.0313, 0.0604, derived from the released per-seed artifacts]. The arms satisfy the §5 descriptive band criterion — each arm's median lies within the other arm's per-seed 10th/90th interval — and we make no lower-exploitability claim. The criterion is a descriptive band check at n=5, not an equivalence test; equivalence testing in the two one-sided tests (TOST) style is deferred to the pre-registered full-protocol G5 run.
- **Secondary, threshold-sensitive view (time-to-band).** A time-to-band τ-sweep ($\tau \in [0.05, 0.12]$) — where time-to-band(τ) is the cumulative inner-resolve-plus-net-train wall-clock (measurement phase excluded) to the first measurement round whose on-policy NashConv is $\leq \tau$ — gives speedups from **5.5× to 11.4×** depending on the threshold; at the pooled band ceiling $\tau = 0.0598$, the median time-to-band is fused 34.7 s vs sequential 190.2 s (ratio of the per-arm medians, 5.48×; these are time-to-band medians — the totals behind the 9.99× headline are the 76.8/767.2 s above).[2] We disclose the full range and keep the headline on the threshold-free total-wall number, 9.99×.
- **Substrate scope (best-vs-best).** The 9.99× is the same-substrate fused-vs-sequential comparison that isolates the mechanism (§4.6). Against the strongest measured *non-fused* cell in the released bundle — the same kernel sequential on the host CPU — the per-belief 2×2 of §4.6 puts the G4 best-vs-best win at ∼2.7× (GPU-fused 0.201 s/belief vs CPU-sequential 0.547 s/belief), pending matched full-configuration CPU timings (single-run CPU cells; §4.6, Appendix C); the same-substrate tooling question — the CUDA-Graphs/`torch.compile` sequential arm — is

[2] The non-monotone dip at $\tau = 0.06$ is a quantization artifact of the every-2-rounds measurement cadence, not a performance effect; the full sweep, the threshold definition, and the explanation are in Appendix D.1.

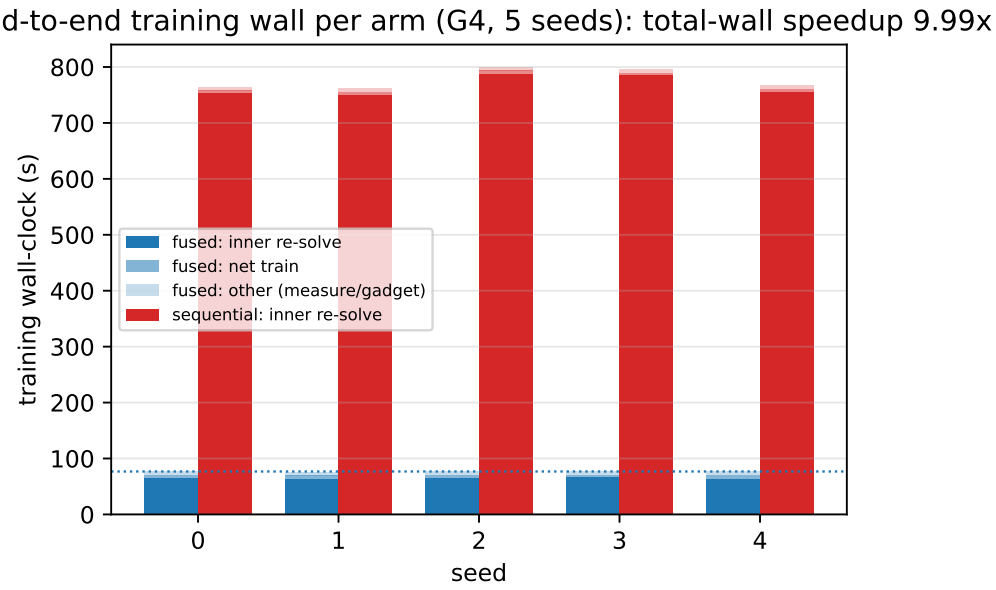

Figure 4: The end-to-end win, per seed. Training wall-clock per arm at Goofspiel-4, stacked by phase (inner re-solve / net train / other, where "other" is the separately-tracked measurement phase; the gadget pass runs after the timed loop and is outside all phase timers), for all 5 matched seeds. The inner re-solve is 98.4% of the sequential arm — the Amdahl fraction that lets the 11.79× inner speedup survive as a 9.99× total-wall speedup (dotted line: fused median).

measured in §6.5. In these runs, in the launch-bound regime, fusion is what made the GPU the faster substrate at all.

- **Gates.** CPU exact parity $< 10^{-9}$; CPU determinism — fused and sequential training trajectories identical to all recorded digits (reduced 4×10 budget; §5); on CUDA, compounding `index_add_` atomics produce per-seed fused-vs-sequential final-NashConv differences of median 0.0144 (max 0.0389) with no detectable bias at this seed count (fused higher on 2/5, lower on 3/5; n=5 is underpowered to exclude moderate bias, and pooled with the quiet-host G5 pairs the tally is fused-higher on 4/8) — the §5 finding that per-seed equivalence checks of batched GPU CFR through a training loop will spuriously fail. Mild host sensitivity is visible even in these clean G4 runs, at much smaller scale than the G5 event (Appendix B.3).

6.3 Search-free boundary (directional only)

Neural Fictitious Self-Play (NFSP) [20] learns without any decision-time or train-time search: it trains a best-response network against an *average-policy* network that imitates the agent's own historical play, and it is that averaged policy — the quantity that converges toward equilibrium — that we evaluate (the green curve of Figure 6). To calibrate what the resolving apparatus buys per unit compute — not to claim a converged comparison — we run NFSP in average-policy mode under one published configuration (§5; chosen over a tuned PPO-with-averaging baseline for the reasons given in Appendix D.2 [47]). After 600 s (∼640k/629k episodes on seeds 0/1), NFSP's exact NashConv is **1.3968 / 1.3966**, barely below the uniform policy's 1.4167; the fused resolving arm reaches its ∼0.04 final band in ∼77 s total — roughly 1/8th of that wall-clock, to a band NFSP has not approached. This is a *directional boundary only*: NFSP's literature operating

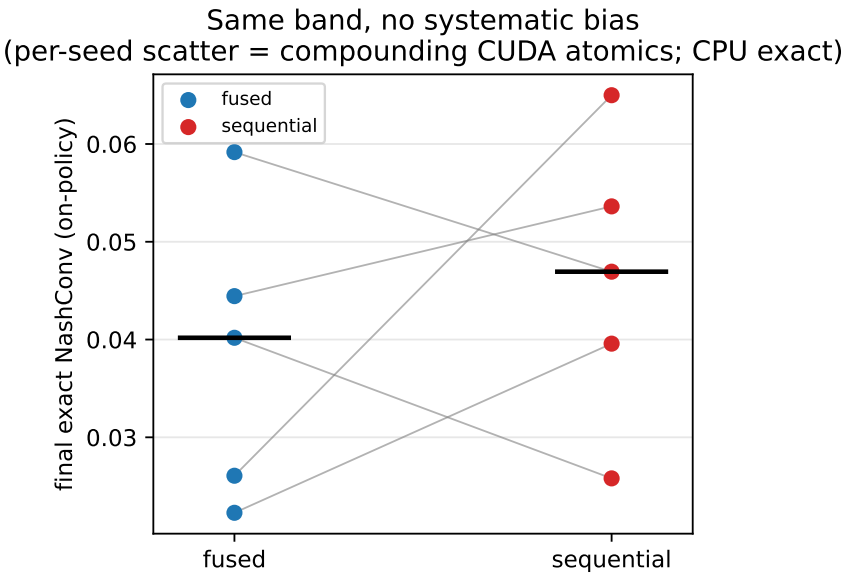

Figure 5: Descriptive band check; no detectable bias (n=5). Final exact NashConv (on-policy continuation) of both arms at Goofspiel-4, per seed, with paired per-seed lines and medians (black bars): fused 0.0402 vs sequential 0.0469. The per-seed scatter (median |diff| 0.0144, max 0.0389; fused higher on 2/5, lower on 3/5) is the compounding-CUDA-atomics effect of §5 — algorithmic equivalence is proven on CPU, where trajectories are identical to all recorded digits; the seed-sign tally is consistency evidence, not proof of an unbiased CUDA drift process.

point is 10^6–10^7 episodes, and we report it to locate the resolving loop on the compute axis, not to rank methods. The accompanying methodology point (detail in Appendix D.2): RPG/QPG current-policy NashConv oscillates at ≈1.4334 — *above* uniform — so reporting it as the search-free boundary would artificially inflate our advantage, and we reject it; VRPO is cited as the strongest search-free boundary we are aware of and deliberately not reimplemented.

6.4 Goofspiel-5 at scale: the 5-seed band and the saturated-regime end-to-end point

(a) The 5-seed exploitability band — a target-construction result. Scope first: this result is about what fusion makes *affordable* — per-belief finite-budget CFR+ targets at 236k infosets — and it compares two *target constructions*; it is **not** a fused-vs-sequential comparison, and the arms comparison everywhere in this paper remains equal-band only. The reference point is the **0.21 fixed-continuation floor**: in an earlier experiment under the same implementation and metric (artifacts in the release, Appendix F), the value net was trained on targets computed under a *fixed* below-cut continuation policy rather than per-belief re-solving, and G5 exploitability plateaued at ≈0.21 regardless of additional net training. Why a floor arises: targets computed under one frozen continuation are *incoherent off-path* — they answer "what is this state worth if play continues in a way that no policy consistent with the belief actually plays" — so the value net regresses toward numbers that no assembled policy attains, and the loop's achievable exploitability is capped by this target incoherence rather than by optimization effort (a *target-coherence* ceiling; value-error-to-exploitability propagation in the sense of [27]). Per-belief V_T targets — affordable at G5 precisely because the fused solver cuts the cost of the K=15-key population re-solve — remove the incoherence. Under the identical advance-fixed protocol (§5 full configuration; 320 resolved

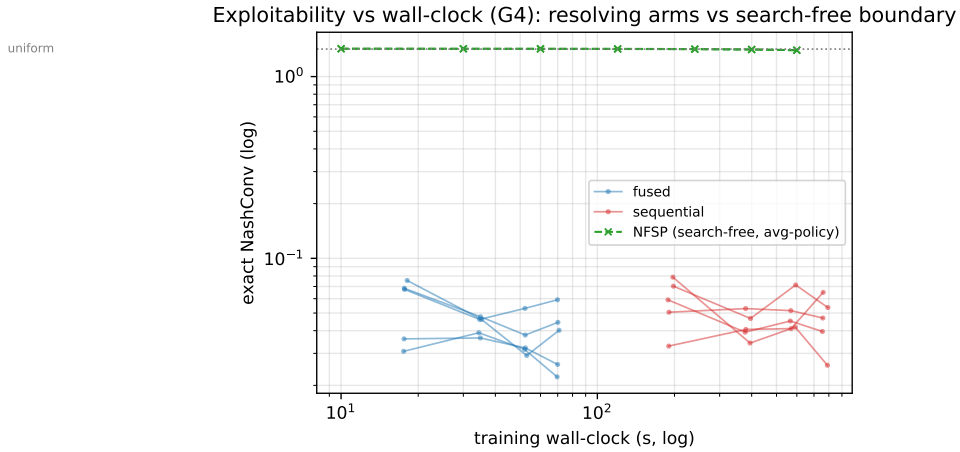

Figure 6: Exploitability vs wall-clock (Goofspiel-4, log–log). Exact NashConv against cumulative training wall-clock: fused arm (blue) and sequential arm (red), 5 seeds each, with the NFSP average-policy curve (green, dashed) and the uniform-policy level (dotted). Both resolving arms descend to the descriptive band — fused ∼10× earlier; NFSP remains near uniform for its full 600 s budget (directional boundary only).

Table 7: Goofspiel-5 per-belief finite-budget target band (full 8-round protocol, fused path).

seed	on-policy NashConv	gadget-safe NashConv	final target-net val MAE
0	0.0760	0.0880	5.8%
1	0.1133	0.1228	6.1%
2	0.1282	0.1380	6.1%
3	0.0923	0.1010	6.0%
4	0.1015	0.1104	6.1%

beliefs per seed; exact NashConv on the assembled policy), the five seeds give:

- **On-policy band: median 0.1015 [10th/90th: 0.0825, 0.1222].** Gadget-safe band: median 0.1104 [10th/90th: 0.0932, 0.1319] — reported alongside per §5, with band claims on the on-policy continuation.
- **All 5/5 seeds finish below the 0.21 fixed-continuation floor**; the median improvement over the floor is **2.07**×. As delimited above, this is a target-construction comparison — what fusion makes affordable — orthogonal to fusion itself; it is **not** a fused-vs-sequential quality claim.
- The final target-net validation MAE is tight across seeds (5.8–6.1%), and we find **no reproducible improves-with-target-compute trend below ∼0.13** (across the 5 band seeds and the earlier seed-0/1 curve runs; Appendix F) — initialization and optimization noise dominate there, which is why this is a band with percentile error bars and not a curve, and why all speed claims in this paper are equal-band-faster.

(b) The saturated-regime end-to-end point. The §6.1 model predicts that at G5 — where a single subgame fills the device — the in-loop inner-resolve speedup should fall from G4's ∼11.7× to the saturated plateau, microbenchmark value 2.37×. We test this with the same paired-arm design at G5 (shortened budget per §5: 4 rounds × 40 beliefs, single final exact measurement, gadget evaluation skipped). The headline numbers are the **quiet-host re-run**: 3 matched seeds

× both arms on an otherwise idle machine, run after a host-load contamination diagnosis in the original 2-seed pair (summarized below; full forensics in Appendix B.3).

- **Inner-resolve speedup 2.40× (median of per-seed ratios, n=3, quiet host; per-seed 2.373/2.400/2.398)** — within ∼1.1% of the microbenchmark prediction of 2.37× (unrounded: 2.398 vs 2.372). Together with §6.2 (G4: ∼11.7× predicted → 11.79× measured, n=5), the regime model's microbenchmark-to-loop transfer holds at both ends.
- **The original 2-seed pair is superseded by the quiet-host re-run, but reported as host-load sensitivity evidence.** The diagnostic that exposed it: the pair's total-wall ratio (2.20×) *exceeded* the 2.04× Amdahl ceiling implied by its own measured inner fraction — by the §6.2 decomposition this requires a ΔO an order of magnitude beyond the noise scale that identical non-inner work can produce. The per-round artifacts located the cause: host-load-sensitive GPU-event spans in the launch-heavy *sequential* arm (the §5 timing-discipline mechanism — the fused arm, issuing ∼15× fewer launch sequences per belief, stayed fed). The quiet-host n=3 re-run supersedes the pair — prediction 2.37×, measurement 2.40× — and restores the self-consistency check the contaminated pair failed (next bullets). Per-round event spans, the inflation figures, the contaminated pair's face-value readings, and the ΔO accounting applied to it are in Appendix B.3.
- The inner re-solve is **86.1%** of the sequential loop (median, n=3, quiet host; below G4's 98.4% because the single final measurement is a larger share of the shortened run).
- **Total-wall speedup 2.00× (median of per-seed ratios; per-seed 1.99/2.00/2.00), against the 2.01× Amdahl ceiling** implied by the measured 86.1% inner fraction and the 2.40× inner ratio — the accounting now closes to within 0.3% (computed from unrounded artifact values, as in §6.2; the quiet-host re-run itself realizes one sub-percent per-seed ceiling excess, which the ΔO reading accommodates), the same closure as G4's, where the contaminated pair had violated its own ceiling. A time-to-band τ-sweep is flat at 2.39× across $\tau \in [0.05, 0.12]$: with a single final measurement, time-to-band coincides with the training wall-clock, so the G4-style cadence sensitivity (§6.2) cannot arise here.
- **On final exploitability we make a divergence-scale-consistency statement, not a band claim.** Per-seed finals (fused, sequential): seed 0 **(0.1543, 0.0912)**, seed 1 **(0.0974, 0.0857)**, seed 2 **(0.1243, 0.1457)**. The fused final is higher (worse) on **2/3** seeds — a direction n=3 cannot assess, which we state rather than imply away. The per-seed divergences (0.0631/0.0117/0.0214) are consistent in scale with the compounding-CUDA-atomics envelope documented at G4 (median 0.0144, max 0.0389; the G5 max exceeds it ∼1.6×) — not inconsistent with the §5 mechanism, though the envelope's scaling with game depth is uncharacterized, which the planned full-protocol G5 run will bound. These shortened-protocol (4-round) finals are *not comparable* to the 8-round 5-seed band above in either direction, and two of the six lie outside that band's observed range — so no band-membership claim is available from these runs, and we make none. Through-training equivalence at G5 rests on the CPU gates and the single-solve parity (4×10^{-6}); establishing statistical CUDA non-divergence at G5 requires the planned full-protocol $n \geq 5$ paired run with a pre-registered margin. No lower-exploitability claim is made in either direction.
- The safe-resolving gadget is outside all timing claims: it runs after the timed loop, outside every phase timer and outside `total_wall` (§5). The quiet-host paired G5 runs skip it uniformly; placement notes and the superseded pair's single post-loop gadget reading are in Appendix D.4.

The two G5 results compose into the paper's working point: the fused solver makes per-belief

finite-budget CFR+ targets affordable at 236k infosets (the band above), and even in the regime where fusion pays *least*, it still pays the predicted ∼2.4× on this device — with the prediction made by a model fit on microbenchmarks alone.

6.5 Launch-amortization ablation: the headline comparison under tool symmetry

§4.6 identified CUDA-Graphs capture and `torch.compile(mode="reduce-overhead")` as the rival lever and promised the measured ablation; here it is, in both directions. Six arms — {fused, sequential} × {eager dispatch, compiled, graph-captured} — on the §6.1 protocol (300 CFR+ iterations, median of 7 repeats after 3 discarded warmups, float64, precompiled topologies; compile/capture setup walls excluded symmetrically and reported in the artifact: `torch.compile` ≤9.3 s per game, graph capture ≤0.15 s). The graph-captured arms replay an even/odd CUDA-Graph pair executing the *same* kernel sequence as eager — pure launch amortization — while the compiled arms additionally re-codegen (fuse) the per-iteration kernels; the two tools therefore separate launch overhead from eager-ATen kernel inefficiency (capture engineering in Appendix C.1). Parity is gated as everywhere: at per-tree granularity the tooled sequential arms match the eager reference to $\leq 3.3 \times 10^{-11}$ at G3/G4 (gate 10^{-9}); at fused granularity G3/G4 pass with over 2.5 orders of margin (compile 3.0×10^{-12}, graphs 5.4×10^{-14} at G4, against an eager-rerun atomics floor of 1.4×10^{-12}); at G5 the gate is **floor-limited**: the eager fused reference differs from *itself* by $\sim 2 \times 10^{-8}$ between reruns (CUDA `index_add_` atomics over B=125 cut nodes × 236k infosets compounded through the 200-iteration parity run; reproduced at 5.7×10^{-9}–1.7×10^{-8} across independent rerun pairs), and the tooled arms sit at 1.00–1.05× that floor — indistinguishable from an eager rerun, with no evidence of a numerics change, but not certifiable to 10^{-9}. G5 rows below carry this caveat.

Step 1 — the asymmetric comparison (the natural first comparison). Amortize the *sequential* arm only, and compare it against the eager-dispatched fused solver. Launch amortization is real: it lowers the sequential per-tree floor by 72%/66% (compile/graphs) at G4 and 63%/32% at G5. But the amortized sequential arm *remains linear in* N (G4 compile slope 56.82 ms/tree, vs 201.09 eager) while the eager fused arm stays flat, so the fused advantage is rescaled, not removed: the surviving best-vs-best speedups are **2.59× (G3), 3.28× (G4), 1.03× (G5)**, with the G4 crossover moving from $N^* \approx 1$ to $N^* \approx 3.63$ and the retained advantage still growing as $\sim N/3.63$. The G5 reading decomposes exactly as §4.6 predicted for capture — pure graphs recover only 1.31× of the sequential arm's cost and the eager fused arm retains 1.85× over them — but `torch.compile` goes further: its within-tree kernel re-codegen brings the compiled *sequential* arm to parity with the eager-ATen *fused* arm (1.03×). Taken alone, this table understates the mechanism, because it grants amortization tooling to one arm and not the other; we report it because it is the comparison this setup produces first, and because the asymmetry is exactly what motivates step 2.

Step 2 — the symmetric comparison (best tool per arm). The identical tooling applies to the fused topology unchanged — the same even/odd-pair and compiled-step machinery at population granularity — and the arms then coincide at N=1 by construction (G4: 55.2 vs 56.6 ms; G5: 109.6 vs 111.1 ms), so everything beyond N=1 is the fusion mechanism itself (Table 8).

The sequential arms and eager-vs-eager reference come from the per-key session; the amortized fused arms were measured in an independent session with the fused-eager arm re-measured as the cross-session drift anchor (ratios 0.969–0.998). The independent session is ≤3% faster, so the best-vs-best ratios carry ≤3% cross-session uncertainty in fusion's favor; no conclusion is within that margin except the N=1 coincidence, which holds by construction. The ablation's own eager-vs-eager column (8.82/11.64/2.42×) reproduces Table 6 (8.77/11.69/2.37×) within ∼2%.

Table 8: Six-arm launch-amortization ablation (full cut population, 300 CFR+ iterations; ms, GPU-event medians; best amortized arm per side in bold).

	fused			sequential			amortized
game	eager	compile	graphs	eager	compile	graphs	best-vs-best
G3 (N=9)	117.1	**39.1**	40.0	1,059.4	355.5	**311.7**	**7.96**×
G4 (N=12)	202.1	**56.6**	85.5	2,426.7	**683.0**	810.2	**12.06**×
G5 (N=15)	1,871.0	**1,060.3**	1,817.4	4,536.2	**1,929.2**	3,465.6	**1.82**×

- **Launch-bound regime (G4): the fusion advantage is speedup $\approx N$, with or without tooling.** The amortized G4 key sweep gives amortized-fused over amortized-sequential **1.02/1.97/3.97/7.84/11.62× at** $N = 1/2/4/8/12$ — speedup $\approx N$ at every point, the same shape as the eager-vs-eager sweep of §6.1. The amortized fused arm is flat (55.2 → 58.7 ms for N: 1→12 — slope 0.31 ms/tree on a 55.2 ms floor, +6.2% for 12× the population) while the amortized sequential arm stays linear, and step 1's $N^* \approx 3.63$ crossover collapses back to ≈1. The two levers compose multiplicatively: compile is worth 3.57× on the fused arm at G4, on top of fusion's $\approx N$ over sequential.
- **Saturated regime (G5): step 1's parity verdict reverses to 1.82×.** Pure capture buys essentially nothing on the fused arm (fused-graphs 1.03× over fused-eager): the fused G5 solve is genuinely compute/occupancy-bound, confirming the §6.1 saturation story. Compile's kernel re-codegen still buys 1.76×, and under compile-vs-compile the *marginal* cost per additional tree is 68.0 ms/tree fused vs 129.6 ms/tree sequential — so the amortized-vs-amortized speedup rises monotonically (1.01/1.34/1.60/1.71/1.81× at $N = 1/2/4/8/15$; 1.82× on the cross-game row) toward the marginal-slope ratio ≈1.9, with no crossover: dominance from N=1. Step 1's G5 parity was an artifact of comparing eager-ATen fused kernels against compiled sequential ones.
- **Regime-model upgrade.** With per-arm floors and slopes, the §6.1 envelope refines to $\text{speedup}(N) \approx (\text{floor} + \text{slope}_{\text{seq}}(N-1))/(\text{floor} + \text{slope}_{\text{fused}}(N-1))$, with $\text{floor}_{\text{fused}} \approx \text{floor}_{\text{seq}}$ under the same tooling: $\to N$ when $\text{slope}_{\text{fused}} \approx 0$ (launch-bound; measured 11.62× at N=12), $\to \text{slope}_{\text{seq}}/\text{slope}_{\text{fused}} = 129.6/68.0 \approx 1.9$ at saturation (measured 1.81× at N=15). Launch amortization rescales both arms' floors equally; it cannot touch the cross-tree occupancy term. The fusion advantage *is* the marginal-cost ratio, and it survives.
- **Tool-symmetric end-to-end check (G4).** We reran the full G4 loop with `torch.compile` on both arms using 5 matched seeds and the same aggregation discipline as §6.2. Fused and sequential pass the same broad descriptive band check (pooled ceiling $\tau = 0.0506$; final medians 0.0475 fused vs 0.0304 sequential; fused reaches the pooled-ceiling band in 11.0 s train wall, sequential in 109.9 s), and the fused arm is faster by 11.36× on inner resolve, 7.38× on total wall, and 9.99× time-to-band against a 7.39× Amdahl ceiling for total-wall speedup. This is a timing result under a coarse quality gate, not a lower-exploitability result. The remaining tool-symmetric end-to-end extension is the analogous G5/production-scale paired CUDA band run.

7 Related Work

We position the lever inside the cost identity of Eq. (1), then delimit it from the nearest structural neighbors. Non-claim-bearing expansions are in Appendix E.

The lever taxonomy of Eq. (1). The wall-clock cost of train-time iterated resolving factors as (number of solves) × (iterations per solve) × (wall-clock per CFR iteration of a solve), with sampling variance per traversal a further axis when traversals are stochastic. Prior work attacks the first two factors and the sampling axis; to our knowledge no published work claims the **cost-per-solve** factor for *populations* of subgames — the lever this paper isolates.

- **Number of solves (N_{solves}).** TurboReBeL [35] is the nearest cost-efficiency cousin of this paper: it reports matching ReBeL's exploitability at a small fraction of its training cost by amortizing target generation across CFR iterations — its primary lever is **single-sample multi-iteration generation** (one sampling pass yields data for all T iterations), with suit/chip isomorphism augmentation as a secondary, poker-specific lever. It reduces the number of solve/sampling passes per training datum; each solve is still executed one subgame at a time. The levers compose: theirs is per-datum solve count, ours is per-solve cost.
- **Iterations per solve.** DCFR [9] and PCFR+ [18] accelerate the convergence of a single solve by discounting and prediction. Meta-learned predictive regret minimization — originated in [52] and extended to self-play in [53] — trains over a *distribution* of subgames but deploys on each subgame *sequentially*: it reduces iterations per solve, not the cost of an iteration, and stacks with our lever. Deep-Predictive DCFR [58] — building on Deep CFR [10], the canonical neural CFR — and AlphaEvolve-discovered CFR variants [37] extend this line (Appendix E).
- **Samples per traversal.** MCCFR [31] and its variance-reduced descendants VR-MCCFR [48], ESCHER [40], and DREAM [51] reduce the variance (hence the count) of sampled traversals; the per-iteration arithmetic of each solve is untouched.
- **Subgame size.** LAMIR [29] learns abstractions that limit each resolved subgame to a manageable size, making principled look-ahead tractable in games where it previously could not scale. Smaller subgames *deepen* the launch-bound regime in which fusion pays most (regime model, §6.1).
- **Cost per solve (this work).** Fusing the population of same-topology depth-limited PBS subgames generated by one resolving sweep into shared batched kernel launches, with parity to per-tree CFR+ gated as in §5.

GPU-accelerated CFR. Kim [24] vectorizes CFR *within* a single game tree, reporting speedups of up to 401.2× over OpenSpiel's Python CFR and up to 203.6× over its C++ implementation; our sequential baseline arm has exactly this shape (the same level-grouped SoA kernel, applied one subgame at a time). Kim & Sandholm [25] reframe CFR as a series of linear-algebra operations — a sequence-form-as-linear-algebra formalization with proven work–depth bounds for the parallelized iteration (total work $W = \Theta(|P|)$ at depth $D = \Theta(k \log B)$, in their notation) and measured speedups of up to four orders of magnitude over OpenSpiel's CPU implementations — and the authors' NoRegret library (v0.0.0.dev15, released 2026-06-04) makes this an actively developed, still per-tree line; a GPU-CFR implementation for the card game Pasur [4] and the open-source gpucfr code (CTU Prague) [46] are further per-tree/per-game data points. Li and Huang's Real-Time Parallel CFR [36] is the newest adjacent HUNL systems result we found: it parallelizes a single depth-limited CFR solve through CPU/GPU pipeline stages, pruning, abstraction, and GPU leaf evaluation. That is orthogonal to the lever isolated here, which fuses a population of same-topology solves generated by one resolving sweep into shared kernel launches. As of 2026-06-11 we find no public work claiming population-level fusion on this line. We note plainly that in the linear-algebra formulation of [25], a *mechanical* multi-tree batch dimension is an anticipatable engineering extension — a block-diagonal arrangement NoRegret could ship in a point release. What this paper claims is therefore the conjunction, not the batch dimension: the identification that *iterated resolving*

structurally generates the population, the parity-gated loop integration with shared range-vector state, and the predictive regime model with its end-to-end validation. Those survive even if per-tree libraries add batching.

Batched solving and adjacent claims in game solving. Six nearby claims are distinct from ours. (i) Marris et al. [39] batch equilibrium solving, but with a *learned, amortized* solver over *normal-form payoff matrices*; we run parity-gated CFR+ over *extensive-form* depth-limited PBS subgames — variable depth, chance nodes, per-subgame regret and range state — inside a training loop. (ii) DeepStack's batching is net-eval batching, not solve batching. (iii) The distribution-of-subgames in [53] is a meta-training set for a predictor that is then applied sequentially per subgame. (iv) US Patent 11,204,803 [34] decomposes the game into independently solvable subtasks — task-level parallelism, not fusion of many solves into shared kernel launches on one device. (v) The within-public-state range dimension — solving for all private hands simultaneously — is classical [22, 23] and is present in *both* of our timing arms; our claim is the across-key axis (§2.2, §4.5). (vi) Concurrent subgame solving per se has distributed-CPU precedent: Cepheus, which essentially solved heads-up limit hold'em, decomposed the game into a trunk and subgames solved concurrently across distributed CPU nodes [5, 55]; what is unclaimed is the single-device GEMM-fusion of the subgame population into one kernel-launch sequence.

Continual resolving and decision-time search. DeepStack [43] introduced continual re-solving with neural counterfactual-value leaves; what it batches are *net evaluations* at subgame leaves, not the solves themselves. Naive depth-limited solving is unsound [8]; safe-resolving gadgets [13, 42, 43] and safe-and-nested subgame solving [6] restore soundness (the background is carried by §2.2). ReBeL [12] moved resolving into the training loop on public belief states — the loop whose inner solve we fuse — and Student of Games [49] unified search-based agents across game classes; within this line, TurboReBeL and LAMIR attack N_{solves} and subgame size respectively (taxonomy above). Kubicek, Lisy & Sandholm [30] show that resolving-gadget equilibria are non-unique and refinement-sensitive; accordingly, our exploitability-band claims use the on-policy re-solve continuation, with gadget numbers reported alongside (§5). The agent lineage (Libratus, Pluribus) and opponent-model resolving (CDBR) are discussed in Appendix E.

Systems prior art: batching small workloads. The batching economics themselves are classical and we do not claim them: §2.3 reviews the batched-BLAS insight and the roofline frame, and the FlashAttention template we follow is credited in §1. CUDA Graphs [44] amortize launch overhead for a fixed kernel sequence without fusing work — the rival lever, analyzed for our kernel in §4.6. In the games-adjacent systems literature, mctx [15] batches MCTS search populations in the perfect-information family; Pgx [28], EnvPool [56], and Isaac Gym [38] vectorize game/environment *simulation*, not iterative coupled-state solving (distinctions in Appendix E). None of these carries a shared cross-instance regret table through a clamped fixed-point solve under an exactness gate — that conjunction, in the extensive-form resolving loop, is what is new here; the batching pattern itself is not, and we do not claim it.

Search-free self-play. NFSP [20] and regret/Q-based policy gradients (RPG/QPG) [50] learn without decision-time search; our directional search-free boundary uses NFSP's *average* policy, since evaluating the RPG/QPG *current* policy early in training is near-uniform and would be a weak comparison (§6.3, Appendix D.2). VRPO [17] reports a positive win rate against Slumbot in HUNL — a result its authors describe as not a decisive win — without online search or subgame solving, and is the strongest search-free line we are aware of; we cite rather than reimplement it, because the primary baseline for a cost-per-solve claim is the in-experiment sequential arm sharing the same kernel, the same seeds, and gated parity.

8 Limitations, Scope, and Future Work

Scope of the demonstrated loop. Our end-to-end demonstration is a **2-level, single-cut** iterated-resolving loop: the trunk is re-solved against value targets obtained by re-solving each cut subgame at sampled beliefs, and these per-belief targets are *independent given the belief*. This independence is exactly what makes the population of subgames fusable, so the obvious objection — surfaced at the end of §4 — must be answered plainly: *is this just batching embarrassingly parallel work?* Our rebuttal is a conjunction, and we do not claim any single ingredient is novel in isolation: (a) identifying that train-time iterated resolving *structurally generates* a population of same-topology depth-limited PBS subgames — a common and important target-generation workload in this lineage (DeepStack's 10^7 independently re-solved subgames, §2.2); (b) a fused CFR+ whose nontrivial content is the level-grouped compilation of irregular structure, float64 discipline through compounding fixed-point iterations, and live-loop integration, with parity to the sequential solver gated rather than assumed (§4.5 — the within-key coupled state is classical [22, 23] and not the novelty); (c) a predictive floor/slope regime model, $C_{\text{seq}} = a_s + b_s(N-1)$ and $C_{\text{fus}} = a_f + b_f(N-1)$, with the $\min(N, 1/\varphi)$ expression retained as the launch-bound envelope and with an operationalized two-point marginal-cost probe (§6.1); and (d) an Amdahl-surviving end-to-end result (the ex-ante inner prediction 11.7× realized at 11.79×; total-wall accounting closing to within 1% of the 10.09× ceiling). A relevant structural distinction for what lies beyond: DeepStack-style *independent target generation* is fully fusable, while ReBeL-style *episodic* resolving is serially coupled along an episode (fusable across parallel episodes and keys, not along the chain); and the belief axis is a second fusion dimension — our loop already solves 40 sampled beliefs per round as 40 separate fused solves over a belief-independent topology, so beliefs can be folded into the population to restore the fused width when level-coupling shrinks K. The strongest form of the claim — fused-vs-sequential **timing under ≥3-level nested resolving**, where targets become serially coupled across levels — is deferred to future work. Multi-level self-play *correctness* on this implementation is already certified (Goofspiel-4: exploitability decreases 0.0506 → 0.0145 with training; recorded in the supplementary measurement notes, Appendix F; a standalone artifact is planned); only the multi-level timing comparison is outstanding.

Baseline tooling and substrate. The §6.2/§6.4 end-to-end speedups are measured against an eager-dispatch, single-stream sequential arm. The CUDA-Graphs/`torch.compile` tooling question is now measured at the cost-per-solve level, in both directions (§6.5, Appendix C.1): under tool symmetry fusion retains speedup $\approx N$ launch-bound (12.06× at G4) and ∼1.8× at saturation. The G4 tool-symmetric end-to-end check is now complete: compiled-vs-compiled fusion gives 11.36× inner and 7.38× total-wall speedup while passing the same broad descriptive band check (final medians 0.0475 fused vs 0.0304 sequential). The remaining item on this axis is the analogous G5/production-scale paired run. Our own CPU-gate artifacts imply that at G4 scale the GPU-sequential arm is ∼4.3× slower per belief than the same kernel on the host CPU — making the cross-substrate best-vs-best G4 win ∼2.7× as currently measurable (single-run CPU cells, no timing discipline; §4.6, Appendix C); matched full-configuration CPU timings (G4 and G5) remain planned. Multi-stream per-tree dispatch and fp32-with-compensated-accumulation are further untested alternatives in the same family.

One game family, one device. All end-to-end timing is on the Goofspiel ladder; Leduc and liar's-dice are parity-tested for the fused solver but not end-to-end timed. All measurements use a single consumer GPU (RTX 3070 Ti, 8 GB) whose FP64 throughput is 1:64 of FP32 — an extreme placement for a float64 workload. The regime model is likewise subject to memory capacity (§6.1): the 8 GB card bounds the fusable population ($B_{\max}$ unmeasured; every population in this paper fits), and peak VRAM per arm is not yet instrumented. The direction of this asymmetry is *favorable*

to the lever: on a data-center part with 1:2 FP64 (A100) and ∼10× the FP64 throughput, both the effective launch floor and the saturation point would change; the model predicts the outcome only after measuring the regime probe on that device. On this device, the both-ends validation and the HUNL-mass argument below hold as measured; their numeric values are device-conditional, and A100 replication is future work.

Exploitability is a band, not a curve. On the largest instance (Goofspiel-5, 236,450 infosets) final exploitability sits in a 0.07–0.13 band where initialization and optimization noise dominate (below ∼0.13 we found no reproducible improves-with-target-compute trend). We therefore make **time-based, equal-band claims only** — never lower-exploitability claims — with both arms sharing seeds. The completed 5-seed G5 band is on-policy median 0.1015 [10th/90th: 0.0825, 0.1222] (gadget-safe median 0.1104 [0.0932, 0.1319]; all 5/5 below the 0.21 fixed-continuation floor, defined in §6.4). The G5 paired end-to-end finals are shortened-protocol values that are not band-comparable and have fused higher on 2/3 seeds (quiet-host re-run); per §6.4 we make a divergence-scale-consistency statement there, not a band claim. Fused-vs-sequential equivalence is proven on CPU (exact parity $< 10^{-9}$; determinism identical to all recorded digits at the reduced gate budget); on CUDA, `index_add_` atomics compound through training into per-seed final-exploitability differences of median 0.0144 (max 0.0389) at G4 with no detectable bias at n=5 (§5); the divergence envelope's scaling with game depth is uncharacterized — the G5 max divergence exceeds the G4 max (§6.4) — and the planned full-protocol G5 run will bound it. Gadget-based safe-resolving numbers additionally inherit refinement-choice sensitivity [30].

The saturated regime is a bound, and we state it as one. When a single subgame occupies the device, fusion degrades to ∼2.4× on this device (Goofspiel-5: microbenchmark 2.37×, measured in the end-to-end loop at 2.40×, n=3 quiet host, §6.4). We regard this as a *contribution* — the cost model predicts where the lever pays (the launch-bound, many-small-subgames regime that iterated resolving structurally generates) and where it does not — but it caps the claim: the ∼10× figure is regime-conditional, not universal. The measured inner fractions are likewise operating-point properties: the 98.4% (G4) and 86.1% (G5) inner shares reflect a small value network (MLP, hidden 128) and a sparse measurement cadence; with production-scale value networks the non-inner share grows, f drops, and the total-wall multiple contracts toward the inner-resolve ratio applied to a smaller fraction — exactly as the Amdahl decomposition of §6.2 prescribes, and in the direction the G5 point already shows ($f = 86.1\%$ gives a 2.01× ceiling).

Subgame-build ceiling. The fused *solver* scales (≈6.2 ms/iter in-loop at Goofspiel-5), but our eager full-enumeration subgame *build* does not: Goofspiel-6 build exceeds 280 s, capping the demonstrated ladder at ∼236k infosets — short of the 10^6–10^7 infosets of the large games evaluated in [47]. Lazy/direct-SoA construction is future work, not a solver limitation. Our scope discipline here matches the one Kim & Sandholm [25] state for their own technique — parallelization "does not necessarily increase the size of the game that can be solved": population fusion likewise raises throughput, not maximum instance size.

Future work. (1) The ≥3-level fused-vs-sequential timing demonstration. (2) The remaining baseline items: matched CPU timings, and the analogous G5/production-scale amortized-fused vs amortized-sequential end-to-end comparison; the quiet-host G5 timing re-run is complete (n=3, §6.4), and what remains on that axis is the full-protocol (8-round) $n \geq 5$ paired run with a pre-registered band margin for a CUDA non-divergence test. (3) A second game family timed end-to-end. (4) A100 replication and multi-GPU sharding — the population axis shards trivially across devices (unlike a single tree), so multi-GPU *extends* the lever rather than competing with it. (5) A HUNL-scale instance built on lazy subgame construction, where held-out Slumbot [21] becomes the transfer validation; consistent with our protocol, Slumbot supplies no training data, targets, or selection signal at any point. A preliminary HUNL topology census suggests many same-topology river classes,

but class size only upper-bounds train-time co-occurrence and the timing probe uses synthetic payoffs; HUNL timing and held-out Slumbot transfer are future work (mechanics in Appendix D.3). (6) Composition with the other levers of Eq. (1): pairing the population axis with TurboReBeL-style N_{solves} reduction, DCFR/PCFR+/meta-learned iteration reduction, or MCCFR-family variance levers is untested — including the interaction risk that predictive or meta-learned minimizers change the per-iteration op mix the compiled kernel assumes. The within-loop sequential arm — the same kernel run per-tree [24] — remains the primary baseline in every experiment.

9 Conclusion

Train-time iterated resolving structurally generates populations of same-topology depth-limited PBS subgames, and **population fusion** — compiling each population into one level-grouped SoA CFR+ — isolates across-solve amortization of the cost-per-solve factor of its cost identity — wall-clock per solve-iteration, hence, at the common iteration budget T, per-solve wall-clock — with parity to per-tree solving verified in float64 and, on CPU, trajectory-identical to all recorded digits. The claim is deliberately narrow: *equal-band-faster* at G4 under a broad descriptive band check, same-substrate against the same kernel run per tree, in a regime the cost model identifies and a two-point marginal-cost probe classifies — near-linear in the number of co-solved subgames while solves are launch-bound (Goofspiel-4: inner 11.79× predicted ex ante at 11.7×; total wall 9.99× against a 10.09× Amdahl ceiling, 5 matched seeds; under tool symmetry, 11.36× inner and 7.38× total wall against a 7.39× Amdahl ceiling), saturating at ∼2.4× when one subgame fills the device (Goofspiel-5: 2.37× predicted, 2.40× measured, 3 matched quiet-host seeds; the statistical CUDA non-divergence test remains pending). The measured launch-amortization ablation shows the per-solve advantage survives CUDA-Graphs/`torch.compile` tool symmetry — speedup $\approx N$ launch-bound (12.06× at G4) and 1.82× at saturation (§6.5). We claim no exploitability improvement, no cross-substrate superiority beyond the ∼2.7× best-vs-best our artifacts currently support at G4 (single-run CPU cells; §4.6), and no generality beyond one game family and one device until the remaining planned runs — matched CPU timings, the G5/production-scale amortized paired run, and multi-device replication — are complete. What we believe travels regardless: the population-fusion lever and its regime model, the matched-seed paired-arm + CPU-exactness-gate evaluation template, and the warning that compounding CUDA atomics make naive per-seed equivalence checks of batched GPU CFR spuriously fail.

Reproducibility Statement

Code, commits, and artifacts. All experiments live in one repository (release accompanies the preprint). The evidence-bearing commits: `c47cc19` — the cost-per-solve microbenchmark and controlled key sweep (produces `costpersolve_bakeoff.json`); `8def574` — the G4 end-to-end paired arms and the NFSP search-free boundary; `1ff6a8e` — the G5 per-belief target band, seeds 0–1; and the final-data runs (post `dd3d148`; release commit pinned at submission) — the G5 band seeds 2–4, the original and quiet-host G5 paired end-to-end runs, the HUNL topology census probe, and the six-arm launch-amortization ablation of §6.5 (produces `cuda_graphs_ablation.json`), plus the G4 tool-symmetric end-to-end check (produces `e2e_amortized_g4_compile_summary.json`). Every JSON cited in this paper is copied into the tracked directory **`release/v1-artifacts/`** with a `README.md` manifest and per-file SHA-256 sums; those files are the reproducibility bundle that accompanies the paper. One-command reproduction lines for every experiment, the seeds inventory and no-sweep statement, the aggregation restatement, the artifact field-naming notes, the

supplementary-measurement-notes inventory, and the hardware/software stack with total-compute accounting are in Appendix F. The play-time evaluation harness under development for the HUNL-scale instance of §8 — outside this paper's claims — uses PokerKit [26] as its game-state engine.

Held-out integrity. No Slumbot (or any external-agent) data, targets, priors, or selection signals are used anywhere in this paper (`uses_slumbot_data=false`). The HUNL topology census probe (§8) uses only the public betting-tree structure under our own abstraction and synthetic shape-faithful payoff matrices for timing — no Slumbot interaction of any kind.

Author Declarations

Views disclaimer. The views and opinions expressed in this paper are solely those of the author and do not necessarily reflect the views, policies, positions, investment opinions, or risk-management practices of Ontario Teachers' Pension Plan, its affiliates, or any other institution with which the author is associated. This paper is for research purposes only and does not constitute investment advice, risk advice, an offer, or a solicitation to buy or sell any financial instrument.

Competing interests. The author declares no competing interests.

Generative AI usage declaration. Generative AI tools, including OpenAI ChatGPT and Codex, were used to assist with manuscript drafting, editing, code navigation, formatting, and verification workflows. The author reviewed, edited, and verified the manuscript, experiments, figures, citations, and code outputs, and remains solely responsible for the paper's content, scientific claims, experimental results, and interpretation.

References

[1] A. Abdelfattah, A. Haidar, S. Tomov, J. Dongarra. Performance, Design, and Autotuning of Batched GEMM for GPUs. *ISC High Performance* 2016.

[2] A. Abdelfattah, T. Costa, J. Dongarra, et al. A Set of Batched Basic Linear Algebra Subprograms and LAPACK Routines. *ACM TOMS* 47(3), 2021 (the Batched BLAS standard).

[3] G. M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *AFIPS Spring Joint Computer Conference* 1967.

[4] S. Baghal. Solving Pasur Using GPU-Accelerated Counterfactual Regret Minimization. arXiv:2508.06559, 2025.

[5] M. Bowling, N. Burch, M. Johanson, O. Tammelin. Heads-up limit hold'em poker is solved. *Science* 347(6218):145–149, 2015.

[6] N. Brown, T. Sandholm. Safe and Nested Subgame Solving for Imperfect-Information Games. *NeurIPS* 2017.

[7] N. Brown, T. Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science* 359(6374):418–424, 2018 (published online December 2017).

[8] N. Brown, T. Sandholm, B. Amos. Depth-Limited Solving for Imperfect-Information Games. *NeurIPS* 2018 (arXiv:1805.08195).

[9] N. Brown, T. Sandholm. Solving Imperfect-Information Games via Discounted Regret Minimization. *AAAI* 2019 (DCFR).

[10] N. Brown, A. Lerer, S. Gross, T. Sandholm. Deep Counterfactual Regret Minimization. *ICML* 2019 (arXiv:1811.00164).

[11] N. Brown, T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365(6456):885–890, 2019.

[12] N. Brown, A. Bakhtin, A. Lerer, Q. Gong. Combining Deep Reinforcement Learning and Search for Imperfect-Information Games (ReBeL). *NeurIPS* 2020 (arXiv:2007.13544).

[13] N. Burch, M. Johanson, M. Bowling. Solving Imperfect Information Games Using Decomposition. *AAAI* 2014.

[14] N. Burch, M. Moravčík, M. Schmid. Revisiting CFR+ and Alternating Updates. *Journal of Artificial Intelligence Research* 64:429–443, 2019.

[15] I. Danihelka, A. Guez, J. Schrittwieser, D. Silver. Policy Improvement by Planning with Gumbel. *ICLR* 2022 (implemented in the mctx library, github.com/google-deepmind/mctx).

[16] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS* 2022.

[17] Z. Fan, G. Farina. GAE Falls Short in Imperfect-Information Self-Play Reinforcement Learning. arXiv:2605.19235, 2026 (introduces VRPO).

[18] G. Farina, C. Kroer, T. Sandholm. Faster Game Solving via Predictive Blackwell Approachability: Connecting Regret Matching and Mirror Descent. *AAAI* 2021 (PCFR+).

[19] S. Hart, A. Mas-Colell. A Simple Adaptive Procedure Leading to Correlated Equilibrium. *Econometrica* 68(5):1127–1150, 2000.

[20] J. Heinrich, D. Silver. Deep Reinforcement Learning from Self-Play in Imperfect-Information Games. arXiv:1603.01121, 2016.

[21] E. G. Jackson. Slumbot NL: Solving Large Games with Counterfactual Regret Minimization Using Sampling and Distributed Processing. *AAAI Workshop on Computer Poker and Imperfect Information*, 2013.

[22] M. Johanson, K. Waugh, M. Bowling, M. Zinkevich. Accelerating Best Response Calculation in Large Extensive Games. *IJCAI* 2011.

[23] M. Johanson, N. Bard, M. Lanctot, R. Gibson, M. Bowling. Efficient Nash Equilibrium Approximation through Monte Carlo Counterfactual Regret Minimization. *AAMAS* 2012.

[24] J. Kim. GPU-Accelerated Counterfactual Regret Minimization. arXiv:2408.14778, 2024.

[25] J. Kim, T. Sandholm. Parallelizing Counterfactual Regret Minimization. arXiv:2605.14277, 2026 (the authors' NoRegret library, v0.0.0.dev15, 2026-06-04).

[26] J. Kim. PokerKit: A Comprehensive Python Library for Fine-Grained Multivariant Poker Game Simulations. *IEEE Transactions on Games* 17(1):32–39, 2025 (arXiv:2308.07327).

[27] V. Kovařík, D. Seitz, V. Lisý, J. Rudolf, S. Sun, K. Ha. Value Functions for Depth-Limited Solving in Zero-Sum Imperfect-Information Games. *Artificial Intelligence* 314:103805, 2023 (arXiv:1906.06412).

[28] S. Koyamada et al. Pgx: Hardware-Accelerated Parallel Game Simulators for Reinforcement Learning. *NeurIPS* 2023 (Datasets and Benchmarks track).

[29] O. Kubíček, V. Lisý. Look-ahead Reasoning with a Learned Model in Imperfect Information Games. *ICLR* 2026 (poster; arXiv:2510.05048). (Introduces the LAMIR algorithm.)

[30] O. Kubíček, V. Lisý, T. Sandholm. Equilibrium Refinements Improve Subgame Solving in Imperfect-Information Games. arXiv:2601.17131, 2026.

[31] M. Lanctot, K. Waugh, M. Zinkevich, M. Bowling. Monte Carlo Sampling for Regret Minimization in Extensive Games. *NeurIPS* 2009.

[32] M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Pérolat, D. Silver, T. Graepel. A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning. *NeurIPS* 2017 (arXiv:1711.00832).

[33] M. Lanctot et al. OpenSpiel: A Framework for Reinforcement Learning in Games. arXiv:1908.09453, 2019.

[34] H. Li, L. Song (Alipay Hangzhou Information Technology Co.). Determining action selection policies of an execution device. US Patent 11,204,803 B2, granted Dec. 21, 2021 (decomposes large imperfect-information games into independently solvable subtasks).

[35] B. Li, L. Huang. TurboReBeL: 250× Accelerated Belief Learning for large Imperfect-Information Extensive-Form Games. OpenReview preprint yMo7Z670f6, 2025.

[36] B. Li, L. Huang. Real-Time Parallel Counterfactual Regret Minimization. arXiv:2605.19928, 2026.

[37] Z. Li, J. Schultz, D. Hennes, M. Lanctot. Discovering Multiagent Learning Algorithms with Large Language Models. arXiv:2602.16928, 2026.

[38] V. Makoviychuk et al. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning. *NeurIPS* 2021 (Datasets and Benchmarks track; arXiv:2108.10470).

[39] L. Marris et al. Turbocharging Solution Concepts: Solving NEs, CEs and CCEs with Neural Equilibrium Solvers. *NeurIPS* 2022 (arXiv:2210.09257).

[40] S. McAleer, G. Farina, M. Lanctot, T. Sandholm. ESCHER: Eschewing Importance Sampling in Games by Computing a History Value Function to Estimate Regret. *ICLR* 2023.

[41] D. Milec, O. Kubíček, V. Lisý. Continual Depth-limited Responses for Computing Counter-strategies in Sequential Games. arXiv:2112.12594, 2021.

[42] M. Moravčík, M. Schmid, K. Ha, M. Hladík, S. Gaukrodger. Refining Subgames in Large Imperfect Information Games. *AAAI* 2016.

[43] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, M. Bowling. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science* 356(6337):508–513, 2017 (arXiv:1701.01724).

[44] NVIDIA. CUDA Graphs. *CUDA C++ Programming Guide* (feature introduced CUDA 10, 2018); A. Gray, Getting Started with CUDA Graphs, NVIDIA Developer Blog, 2019.

[45] S. M. Ross. Goofspiel — The Game of Pure Strategy. *Journal of Applied Probability* 8(3):621–625, 1971.

[46] J. Rudolf. GPUCFR: a parallel CFR implementation in C++/CUDA. Code, CTU Prague, github.com/janrvdolf/gpucfr.

[47] M. Rudolph et al. Reevaluating Policy Gradient Methods for Imperfect-Information Games. *ICLR* 2026 (arXiv:2502.08938).

[48] M. Schmid, N. Burch, M. Lanctot, M. Moravčík, R. Kadlec, M. Bowling. Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines. *AAAI* 2019.

[49] M. Schmid et al. Student of Games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances* 9(46):eadg3256, 2023 (preprint "Player of Games," arXiv:2112.03178).

[50] S. Srinivasan, M. Lanctot, V. Zambaldi, J. Pérolat, K. Tuyls, R. Munos, M. Bowling. Actor-Critic Policy Optimization in Partially Observable Multiagent Environments. *NeurIPS* 2018 (arXiv:1810.09026).

[51] E. Steinberger, A. Lerer, N. Brown. DREAM: Deep Regret Minimization with Advantage Baselines and Model-free Learning. arXiv:2006.10410, 2020.

[52] D. Sychrovský, M. Šustr, E. Davoodi, M. Bowling, M. Lanctot, M. Schmid. Learning not to Regret. *AAAI* 2024 (arXiv:2303.01074).

[53] D. Sychrovský, M. Schmid, M. Šustr, M. Bowling. Meta-Learning in Self-Play Regret Minimization. arXiv:2504.18917, 2025.

[54] O. Tammelin. Solving Large Imperfect Information Games Using CFR+. arXiv:1407.5042, 2014.

[55] O. Tammelin, N. Burch, M. Johanson, M. Bowling. Solving Heads-Up Limit Texas Hold'em. *IJCAI* 2015.

[56] J. Weng et al. EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine. *NeurIPS* 2022 (Datasets and Benchmarks track).

[57] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *CACM* 52(4):65–76, 2009.

[58] H. Xu, K. Li, H. Fu, Q. Fu, J. Xing, J. Cheng. Deep (Predictive) Discounted Counterfactual Regret Minimization. *AAAI* 2026 (arXiv:2511.08174).

[59] M. Zinkevich, M. Johanson, M. Bowling, C. Piccione. Regret Minimization in Games with Incomplete Information. *NeurIPS* 2007.

A Parity — Proposition, Proof, and Exactness Conventions

A.1 Setting and the perfect-recall convention

A game has *perfect recall* if no player ever forgets their own past observations and actions: every state in an infoset shares the acting player's full sequence of own actions and observations up to that point. All games in this paper (the OpenSpiel Goofspiel, Leduc poker, and liar's-dice instantiations) have perfect recall. The proof sketch below uses it exactly once — to make a scatter-*assignment* (as opposed to a scatter-add) well-defined: under perfect recall, every instance of an infoset carries the same own-reach value within one solve, so writing that value by assignment over per-instance ids cannot race against itself with different values.

A.2 Proposition 1, full statement and proof sketch

Well-formed compiled population. The statement below assumes the same contract as §4.5: perfect recall; a public cut; pairwise-disjoint below-cut infoset-row sets $\mathcal{I}_k$ across public keys; identical initial R, S tables; identical action ordering, legal masks, terminal/chance/payoff tensors, and per-instance infoset-id maps between the fused instance and its per-key solo solve; and the same finite CFR+ budget T and update rules in both arms.

Proposition 1 (across-key parity of fused CFR+)**.** *Fix a compiled population with public keys $k \in \{1, \dots, K\}$, cut instances partitioned as $\mathcal{B} = \biguplus_k \mathcal{B}_k$, and below-cut infoset (regret-row) sets $\mathcal{I}_k$ that are pairwise disjoint — as holds whenever each below-cut infoset determines its public key, i.e., the public outcome at the cut is observed by both players. Run (a) the fused solver on all K keys and (b) the same CFR+ kernel on each key alone, with identical entry reaches, iteration budget, alternating-player updates, clamp at zero, linear averaging, and the same masked regret-matching rule (uniform over legal actions when the clamped row sum vanishes — implemented as a fixed threshold (positive-regret row sum $> 10^{-12}$, the same threshold guarding the final average-strategy normalization), identical in both arms). Then, in exact arithmetic, for every iteration t the fused iterates — regrets, current strategies, strategy sums, and all reach and counterfactual-value quantities — restricted to rows $\mathcal{I}_k$ and instances $\mathcal{B}_k$ equal the per-key iterates of key k; in particular both arms return identical average strategies and leaf values. The proposition concerns finite-iteration output parity only; it does not claim faster convergence or lower exploitability.*

Proof sketch. Induction on t over the four steps of the iteration. (i) Regret matching acts row-wise, so equal rows give equal current strategies on $\mathcal{I}_k$. (ii) The forward pass computes each instance's reaches from its own parent's reaches and the parent's strategy (or chance) entries — purely per-instance data, and every instance lies in exactly one key. (iii) The backward pass scatter-adds per-instance counterfactual values into rows indexed by per-instance infoset ids; by disjointness of the $\mathcal{I}_k$, each row receives contributions only from instances of its own key, so the fused scatter restricted to $\mathcal{I}_k$ sums exactly the multiset of addends of key k's solo solve. (iv) The CFR+ update (clamp, alternating-player gating, own-reach-weighted strategy sums, linear averaging) is again row-wise, and padded action slots are masked out of every update and stay zero, so the arms' differing pad widths do not affect legal-slot values. The own-reach weight entering the strategy-sum update is written by a scatter-assignment over per-instance infoset ids rather than an add; under perfect recall (§A.1) every instance of an infoset carries the same own-reach value (an infoset determines its owner's private state and own action sequence, hence, within one solve, the same entry reach and strategy product), so the assignment is well-defined, and by disjointness of the $\mathcal{I}_k$ the fused write restricted to $\mathcal{I}_k$ coincides with key k's. Hence the fused update is the direct sum over keys of the per-key updates. In floating point the two arms therefore sum the same per-row

multisets of addends and can differ only through intra-row accumulation order and op scheduling — which the CPU parity gate bounds at $< 10^{-9}$ and the CUDA runs realize at $0\text{–}7 \times 10^{-6}$ (§5, §6.1). □

A.3 Leaf-value exactness conventions

The normalized leaf-value convention of §2.2 is validated by a round-trip gate: valuing the trunk through the leaf interface, with EV_i taken under the exact full-game continuation, reproduces exact full-game above-cut values to $\sim 10^{-15}$ — and breaks (large error) under the un-normalized convention, which is what makes the gate discriminating. When the opponent entry reach vanishes ($\sum_{h_{-i}} r_{-i}(h_{-i}) = 0$ — possible only for off-path trunk queries, never for the strictly positive Dirichlet-sampled training beliefs of §5), the normalized value is defined as 0; the convention is inert, since any use of such a leaf value carries the same vanishing opponent-reach segment as a factor in its counterfactual weight.

B Timing Methodology and Host-Load Forensics

B.1 Aggregation conventions, fine print

Every fused-vs-sequential end-to-end speedup in §6 is the median of per-seed ratios. The §6.1 microbenchmark speedups are ratios of median-of-7-repeat timings — there is no seed axis in the microbenchmark. The secondary time-to-band speedups (§6.2, §6.4) are ratios of the per-arm median reach times, because per-seed time-to-band ratios are quantized by the every-2-rounds measurement cadence (Appendix D.1). Where $n \leq 2$ we report the per-seed values themselves and do not dress the midpoint as a robust statistic. Ranges such as "5.5–11.4×" round the underlying values (5.48–11.36) to one decimal; all underlying values are in the artifacts. Unless stated otherwise, 10th/90th intervals are NumPy percentile intervals using the default linear interpolation over per-seed values.

B.2 Descriptive band check and determinism-gate fine print

Descriptive band check. Two arms pass the descriptive band check when each arm's median final NashConv lies within the other arm's per-seed 10th/90th interval. **Self-critique.** The criterion is deliberately descriptive and weak: at n=5 each arm's 10th/90th interval spans nearly its entire sample, so median gaps substantially larger than the one observed would also have passed — it certifies the absence of gross divergence, not statistical equivalence. Equivalence testing in the two one-sided tests (TOST) style, with a pre-registered margin, is deferred to the planned full-protocol G5 run (§6.2). **Determinism-gate fine print.** Trajectory values in the CPU determinism gate are recorded to 5 decimal places, and "identical to all recorded digits" means equality at that recorded precision in every recorded field — validation MAE per round and measured exploitability (e.g., per-round exploitability 0.14935/0.14817, identical in both arms). The regret-matching and average-strategy normalization threshold shared by both arms is the 10^{-12} constant of Proposition 1 (Appendix A.2).

B.3 Host-load sensitivity of GPU-event timing: a diagnostic

The mechanism, stated as a finding in §5: GPU-event spans that bracket a host-driven launch loop are host-load-sensitive, because the event pair measures stream-timeline elapsed time *including* gaps where the device waits for host submission. Under host CPU contention, a launch-heavy (sequential) arm's event spans inflate while a fused arm — issuing ∼15× fewer launch sequences per belief at

G5 — stays fed. Consistent with this, the recorded GPU-event totals equal the wall-clock totals to the millisecond in these runs: the events bracket wall-equivalent regions, and event timing does not immunize the launch-heavy arm against host load.

The G5 contamination event (original 2-seed pair, superseded). The diagnostic that exposed it: the pair's total-wall ratio (2.20×) *exceeded* the 2.04× Amdahl ceiling implied by its own measured inner fraction. By the decomposition displayed in §6.2, $T_{\text{seq}}/T_{\text{fus}} = 1/((1-f) + f/S_{\text{inner}} + \Delta O/T_{\text{seq}})$, a seed's total-wall ratio exceeds its own implied ceiling only if its fused arm's non-inner phases ran faster than the sequential arm's ($\Delta O < 0$) — and since the non-inner phases perform identical work in both arms (§5), any such excess can reflect only sub-percent timing noise (the quiet-host re-run itself realizes one such sub-percent per-seed excess, which this reading accommodates). The contaminated pair's excess is an order of magnitude beyond that scale, which is what made it diagnostic; the same sequential seed-1 run also carries an anomalous 41.4 s net-train phase. The per-round artifacts then located the cause: seed 1's *sequential* arm's inner GPU-event times ran 176.9 → 237.1 / 206.3 / 206.5 s (rounds 0–3), inflated 17–34% under concurrent host CPU load, against seed 0's flat baseline (177.8/177.1/177.2/177.1 s) — while the two *fused* arms were identical across seeds to 0.01%. Taken at face value, that pair's n=2 midpoint is 2.57×; the contamination invalidates it, and restricting to the uncontaminated seed gives 2.369× (n=1). The quiet-host n=3 re-run supersedes both — prediction 2.37×, measurement 2.40× — and restores the self-consistency check the contaminated pair failed (§6.4).

G4 tail of the same effect. Mild host sensitivity is visible even in the clean G4 runs, at much smaller scale: the sequential arm's per-seed inner totals spread 5.1% (749–788 s) vs the fused arm's 3.5% — consistent with the launch-loop host-load mechanism above.

C Baseline Engineering Detail

C.1 CUDA Graphs / `torch.compile` capturability audit — confirmed by the measured ablation

We read our own kernel against capturability before measuring. Per compiled topology, every tensor shape in the solver's iteration body is static across iterations and across re-solves (the level-shape groups, the $[n_I, A_{\max}]$ tables, the $[n_{\text{nodes}}, B]$ reach tensors; re-solves rewrite entry-reach values only), and the loop contains no host-side convergence checks or synchronizing `.item()` reads — so there is **no structural barrier to capture**. The audit's two predicted engineering obstacles are exactly the ones the implementation hit, and both resolved with the predicted remedies: (a) the two iteration-indexed Python scalars — the alternating-player update parity (t mod 2) and the linear-averaging weight $w = t{+}1$ — became a pair of parity-alternating (even/odd) CUDA Graphs sharing one memory pool, with w a 0-dim float64 device tensor advanced *in-graph*; (b) the out-of-place regret/strategy-sum rebindings became a functional step writing through `copy_` into static pre-allocated buffers (manual-capture arm) or were delegated to the cudagraph-trees-managed pool (compile arm). One additional contract surfaced and resolved on the compile arm: `torch.compile(mode="reduce-overhead", fullgraph=True, dynamic=False)` compiles the step cleanly at float64 — no graph breaks, no skipped-cudagraphs warnings — but requires the documented cudagraph-trees feedback-loop discipline, `torch.compiler.cudagraph_mark_step_begin()` before each iteration plus cloning the three state outputs before reuse as inputs (without it, recording fails with the overwritten-output error); the three clone kernels per iteration stay inside the timed region as measured tooling overhead. The manual-capture arm replays the *same* kernel sequence as eager — launch amortization only — which is what makes the §6.5 graphs-vs-compile split interpretable: graphs isolate launch overhead, compile adds within-tree kernel re-codegen. The measured outcome (§6.5) confirms the §4.6 pre-

diction for pure capture: graphs close 73–80% of the eager fused-vs-sequential gap at G3/G4 but only 40% at G5 — the launch-gap component, not the occupancy component — and the amortized sequential arm remains linear in the population size; compile, which also re-codegens kernels, closes 98% at G5 and reaches eager-fused parity there, which is why the tool-symmetric comparison of §6.5 is the decisive one. Implementation: `poker_ai/rebel/iig_batched_graphed.py` (the compiled-step and graph-captured solvers), with runners `scripts/run_rebel_cuda_graphs_ablation.py` (per-key sequential arms) and `scripts/run_rebel_cuda_graphs_ablation_fused.py` (population-granularity fused arms); the gated kernel `iig_batched.py` is untouched, dynamo's cache-size limit is raised to 64 in the runner (several distinct shapes per sweep; tooling configuration only), and every arm is parity-gated per §6.5 — with the G5 fused-granularity gate floor-limited at $\sim 2 \times 10^{-8}$ by the eager reference's own atomics rerun noise (the tooled arms are indistinguishable from an eager rerun).

C.2 CPU 2×2 provenance and protocol

The four cells of the per-belief inner-resolve 2×2 at Goofspiel-4 (Table 5, §4.6) have two sources. The CPU cells come from our CPU determinism-gate artifacts (§5), which contain timings of the same two arms on the host CPU at a reduced budget (4 rounds × 10 beliefs; one run per cell; per-belief values are the run's inner-resolve total divided by its 40 resolved beliefs). Per-belief inner cost is budget-invariant, since each belief is an independent re-solve of the same population at the same 300 CFR+ iterations, the default setting; the gate artifacts do not record the iteration count, so the Appendix F reproduction line pins it explicitly. The GPU cells come from the seed-0 full-configuration end-to-end runs of §6.2 (inner-resolve totals divided by their 320 resolved beliefs; seed-0 values, not the §5 median convention). The 2×2: CPU-sequential 0.547 s, CPU-fused 0.42 s, GPU-sequential 2.353 s, GPU-fused 0.201 s — giving the derived readings of §4.6: GPU-sequential ∼4.3× slower per belief than CPU-sequential, CPU fusion worth 1.30×, and GPU-fused vs CPU-sequential ∼2.7×. Caveats, both directions: the CPU cells come from determinism-gate runs, not a dedicated timing protocol (no warmup/repeat discipline), and the two protocols differ, so the ∼2.7× is provisional on matched budgets; matched full-configuration CPU timings at G4 and G5 are planned. At G5 no CPU timings exist; with 236,450 infosets the arithmetic-dominated comparison is expected to invert, but that is untested and we do not claim it.

D Secondary Results and Boundary Details

D.1 Time-to-band τ-sweep detail

Time-to-band(τ) is the cumulative inner-resolve-plus-net-train wall-clock (measurement phase excluded) to the first measurement round whose on-policy NashConv is $\leq \tau$. The τ-sweep over $[0.05, 0.12]$ gives speedups from 5.5× to 11.4× (underlying values 5.48–11.36; rounding per Appendix B.1). The pooled band ceiling $\tau = 0.0598$ is the 90th percentile of the pooled per-seed final NashConv values of both arms — a level both arms reliably attain; the median time-to-band there is fused 34.7 s vs sequential 190.2 s (ratio of the per-arm medians, 5.48×; these are time-to-band medians, not protocol totals — the totals behind the 9.99× headline are 76.8/767.2 s, §6.2). The non-monotone dip at $\tau = 0.06$ (5.48× between 11.36× and 10.78× neighbors) is a quantization artifact of the every-2-rounds measurement cadence: the fused arm first measures below this τ at its round-3 measurement, the sequential arm at round 1. Future runs will measure every round.

D.2 Search-free boundary detail

We choose NFSP over a tuned PPO-with-averaging baseline — which Rudolph et al. [47] suggest can be stronger per unit compute — precisely because NFSP has a published untuned configuration and an explicit averaged policy; the boundary is directional, and tuning any search-free baseline ourselves would invite an unfair-tuning critique in either direction. A methodological note we believe generalizes: regret/quantal policy gradient (RPG/QPG [50]) keep no averaged policy, so their *current-policy* NashConv oscillates near uniform (G4: ≈ 1.4334 vs uniform 1.4167; recorded in the supplementary measurement notes, Appendix F; a standalone artifact is planned) — *above* uniform at our measurement point — so reporting it as "the search-free boundary" would artificially inflate our advantage, and we reject it. The NFSP run executes on the host CPU (the OpenSpiel PyTorch implementation's default device, which our run does not override) while both resolving arms use the GPU — the §6.3 wall-clock comparison therefore crosses platforms, one more reason it is directional only. The pre-registered criterion under which the NFSP baseline would be replaced is recorded in the artifact. VRPO [17] is the strongest search-free line we are aware of and is a cited boundary, deliberately not reimplemented: the primary baseline for a cost-per-solve claim is the in-experiment sequential arm sharing the same kernel, the same seeds, and gated parity.

D.3 HUNL topology census probe: mechanics

The census enumerates the HUNL public betting tree under our 9-action abstraction and groups river configurations into topology classes; class sizes give the co-solvable populations the GPU probe times. The probe's range dimension is 1,081 private hands per player (the river hand set under our abstraction), and the payoff matrices are synthetic but shape-faithful — the probe is timing-only and involves no Slumbot interaction of any kind. The probe step is $N{=}4$, the smallest sweep point above $N{=}1$ for these classes (§6.1). The light/heavy class definitions and all probe numbers are stated once, in §8 (future-work item 5). The caveat that delimits the census, restated: a class-size census is not a per-sweep co-occurrence count — the batch one can actually form at train time is the number of same-topology subgames co-occurring in one resolve sweep, which the census upper-bounds but does not measure.

D.4 Gadget-evaluation placement

The DeepStack-style safe-resolving gadget evaluation runs after the timed loop, outside every phase timer and outside `total_wall` (§5); it is therefore outside all timing claims. The quiet-host paired G5 runs skip it uniformly (`--skip-gadget`). In the superseded original pair, the fused seed-0 run recorded one final gadget evaluation (0.1536) in that post-loop position — outside its timings and conservative with respect to the fused arm.

E Extended Related Work

None of the material here is claim-bearing; the claim-bearing delimitations are all in §7.

Iterations-per-solve, expanded. DCFR [9] and PCFR+ [18] accelerate the convergence of a single solve by discounting and prediction respectively. Meta-learned predictive regret minimization originated in [52], which trains a predictive minimizer over a *distribution* of subgames; NPCFR [53] extends the meta-learning to self-play. Both deploy on each subgame sequentially: they reduce the iterations a solve needs, not the cost of an iteration, and stack with population fusion in form (composition untested; §8). Deep-Predictive DCFR [58] brings the predictive-discounted family to

the neural setting, building on Deep CFR [10], the canonical neural CFR; AlphaEvolve-discovered CFR variants [37] are search-discovered update rules in the same role.

The MCCFR genealogy. MCCFR [31] replaces full tree walks with sampled traversals; VR-MCCFR [48] adds baselines for variance reduction; ESCHER [40] eschews importance sampling by estimating a history value function; DREAM [51] is the model-free deep variant with advantage baselines. All reduce the variance — hence the count — of sampled traversals; the per-iteration arithmetic of each solve is untouched.

Subgame size. LAMIR [29] learns abstractions that limit each resolved subgame to a manageable size, making principled look-ahead tractable in games where it previously could not scale; smaller subgames deepen the launch-bound regime in which fusion pays most (§6.1).

Agent lineage and opponent-model resolving. Libratus [7] is the canonical agent instance of safe-and-nested subgame solving [6], and Pluribus [11] is its multiplayer continuation, outside our two-player zero-sum scope. CDBR [41] resolves against an opponent model rather than to equilibrium.

Per-game GPU-CFR data points. Beyond the line anchored by [24] and [25] (§7), Pasur-GPU [4] reports a GPU-accelerated CFR implementation for the card game Pasur, and gpucfr [46] is an open-source parallel CFR implementation in C++/CUDA (CTU Prague). Both are per-tree/per-game data points; neither claims population-level fusion.

Environment and search vectorization. mctx [15] batches MCTS search populations in the perfect-information family; Pgx [28], EnvPool [56], and Isaac Gym [38] vectorize game/environment *simulation* — stepping many independent environments in lockstep — not iterative coupled-state solving. None of these carries a shared cross-instance regret table through a clamped fixed-point solve under an exactness gate; that conjunction, in the extensive-form resolving loop, is the delimitation stated in §7.

F Reproduction Commands and Artifact Notes

One-command repro lines (defaults encode the single advance-fixed configuration of §5; run artifacts are written under `autoresearch-session/rebel/`):

```
# 6.1 microbenchmark + controlled key sweep (3 warmups discarded, median of 7)
python scripts/run_rebel_costpersolve_bakeoff.py --num-cards 3 4 5 --iters 300 \
    --repeats 7 --warmup 3 --key-sweep 4 5 \
    --output-json autoresearch-session/rebel/costpersolve_bakeoff.json

# 6.2 end-to-end paired arms (per seed s in 0..4; both arms share the seed)
for s in 0 1 2 3 4; do for m in fused sequential; do
  python scripts/run_rebel_b0_perbelief_targets.py --num-cards 4 --seed $s \
      --inner-mode $m \
      --output-json autoresearch-session/rebel/b0_e2e_g4_${m}_seed${s}.json
done; done

# 6.2 aggregation: medians of per-seed ratios + band summary
python scripts/analyze_rebel_e2e_band.py \
    --glob 'autoresearch-session/rebel/b0_e2e_g4_*_seed*.json' \
    --output-json autoresearch-session/rebel/e2e_band_summary_g4.json

# 6.3 NFSP search-free boundary (seeds 0 and 1)
python scripts/run_rebel_searchfree_baseline.py --num-cards 4 --method nfsp \
    --seed 0 --max-seconds 600 \
```

```
    --output-json autoresearch-session/rebel/searchfree_nfsp_g4_seed0.json

# 6.4 G5 5-seed exploitability band (full 8-round configuration; seeds 0..4)
for s in 0 1 2 3 4; do
  python scripts/run_rebel_b0_perbelief_targets.py --num-cards 5 --seed $s \
      --output-json autoresearch-session/rebel/b0_perbelief_g5_curve_seed${s}.json
done   # seeds 0-1 were produced earlier as b0_perbelief_g5_curve{,_seed1}.json

# 6.4 G5 paired end-to-end (saturated regime; shortened 4-round protocol,
# gadget skipped). Headline = quiet-host re-run, seeds 0..2, machine otherwise
# idle; the original seeds-{0,1} pair (b0_e2e_g5_* + e2e_band_summary_g5.json)
# is superseded and retained as the 6.4 contamination diagnostic.
for s in 0 1 2; do for m in fused sequential; do
  python scripts/run_rebel_b0_perbelief_targets.py --num-cards 5 --seed $s \
      --inner-mode $m --n-target-rounds 4 --measure-every 4 --skip-gadget \
      --output-json autoresearch-session/rebel/b0_e2e_g5q_${m}_seed${s}.json
done; done
python scripts/analyze_rebel_e2e_band.py \
    --glob 'autoresearch-session/rebel/b0_e2e_g5q_*_seed*.json' \
    --output-json autoresearch-session/rebel/e2e_band_summary_g5q.json

# 6.5 launch-amortization ablation: sequential arms (per-key tooling) first,
# then the fused-arm completion (population-granularity tooling; merges the
# 'amortized_fused' key into the same JSON). One game per process for
# dynamo/VRAM hygiene.
for g in 3 4 5; do
  python scripts/run_rebel_cuda_graphs_ablation.py --games $g \
      --output-json autoresearch-session/rebel/cuda_graphs_ablation.json
done
for g in 3 4 5; do
  python scripts/run_rebel_cuda_graphs_ablation_fused.py --games $g \
      --output-json autoresearch-session/rebel/cuda_graphs_ablation.json
done

# 6.5 G4 tool-symmetric end-to-end check (per seed s in 0..4; gadget skipped
# uniformly because it is outside timed total_wall; both arms share the seed)
for s in 0 1 2 3 4; do for m in fused-compile sequential-compile; do
  python scripts/run_rebel_b0_perbelief_targets.py --num-cards 4 --seed $s \
      --inner-mode $m --skip-gadget \
      --output-json autoresearch-session/rebel/e2e_amortized_g4_${m}_seed${s}.json
done; done
python scripts/analyze_rebel_e2e_band.py \
    --glob 'autoresearch-session/rebel/e2e_amortized_g4_*compile_seed*.json' \
    --fused-mode fused-compile --sequential-mode sequential-compile \
    --output-json autoresearch-session/rebel/e2e_amortized_g4_compile_summary.json

# Sec. 8 HUNL topology census + GPU timing probe (forward pointer; out of scope)
python scripts/estimate_hunl_topology_buckets.py --gpu-probe \
    --output autoresearch-session/rebel/hunl_topology_census_probe.json

# Figures 3-6 (pure read of the JSONs above)
python scripts/make_rebel_e2e_figures.py

# Exactness gates (CPU). The fused entry point under test is solve_all_keys_soa
```

```
# (poker_ai/rebel/iig_batched.py).
TESTING_SUITE=1 pytest \
  test/unit/test_rebel_iig_batched.py::test_fused_vs_sequential_soa_parity -v
# CPU determinism gate (reduced budget, disclosed in Sec. 5): run the 6.2
# command pair with --device cpu --n-target-rounds 4 --samples-per-round 10
# --subgame-iters 300 (the last flag restates the default, pinning the
# Appendix C per-belief iteration count); trajectories are identical to all
# recorded digits between --inner-mode fused and sequential (artifacts:
# b0_cpucheck_g4_{fused,sequential}.json -- also the source of the Appendix C
# per-belief CPU timing cells).
```

Field naming convention. The fused/sequential reach-median fields in the G4/G5 summary JSONs are **time-to-band medians at τ = the pooled band ceiling** (Appendix D.1), not protocol totals; the field names are retained for artifact stability and documented here and in the artifact README.

Supplementary measurements. A supplementary measurement-notes file in the artifact bundle documents five secondary numbers not yet backed by dedicated JSON: the 7×10^{-6} cross-game parity upper end; RPG/QPG 1.4334; multi-level 0.0506→0.0145; G6 build >280 s; and the ∼0.75 L1 float32-ablation drift of §4.5. These will be reported in standalone machine-readable files in a future artifact version. The in-loop G5 per-iteration figure (≈6.2 ms, §5) is derived from the released end-to-end artifacts (per-round inner-resolve wall-clock over 40 beliefs × 300 iterations per solve). The G5 topology census of §5 (26,773 nodes across 10 level groups, K=15, B=125; 236,450 total infosets, of which 236,440 are below-cut rows in the batched tables) is re-derived directly from the compiled topology, and the derivation command is recorded in the artifact README. The release also includes the CPU determinism-gate artifacts behind the Appendix C 2×2.

Seeds. G4 end-to-end: seeds 0–4, matched pairs (same seed drives both arms). G4 tool-symmetric end-to-end: seeds 0–4, matched pairs. G5 end-to-end: quiet-host re-run seeds 0–2, matched pairs (the headline); the original seeds 0–1 pair is superseded (seed 1's sequential arm contaminated by host load) and retained as the §6.4 diagnostic. NFSP: seeds 0–1. G5 band: seeds 0–4. HUNL GPU probe: seed 0. The §6.5 launch-amortization ablation has no seed axis (median-of-7-repeat timings, like the §6.1 microbenchmark; parity gates at 200 iterations). No other seeds were run; no configuration sweeps were performed (single configuration fixed in advance; §5; the G5 end-to-end budget shortening is disclosed there and in §6.4).

Aggregation and timing. Headline speedups follow the §5 median-of-per-seed-ratios convention; the microbenchmark and time-to-band speedups are the ratio-of-medians exceptions stated there (per-seed values reported directly when $n \leq 2$). GPU timing uses `torch.cuda.Event` pairs with `torch.cuda.synchronize()` barriers — with the §5 caveat that event spans bracketing host-driven launch loops are host-load-sensitive; microbenchmarks discard 3 warmup runs and report the median of 7 repeats on precompiled topologies.

Hardware/software. GPU: one NVIDIA GeForce RTX 3070 Ti (8 GB, FP64 1:64), driver 595.71, PyTorch float64 tensor ops. Host: Intel Core i7-12700K (12 cores / 20 threads), 32 GB RAM, Ubuntu 24.04 LTS. Software: Python 3.13, PyTorch 2.12 (CUDA 13.0 build), OpenSpiel 1.6.15. Exactness gates run the identical code on the host CPU; NFSP runs CPU-only (§5). Total compute: the artifact-recorded GPU runs of this paper — both arms, all seeds, including the superseded contaminated G5 pair, the G5 band seeds 2–4, and the G4 tool-symmetric end-to-end check — sum to ≈4.7 GPU-hours on the single device; the two earliest G5 band runs (seeds 0–1) predate the wall-clock field and are of comparable scale to the later band seeds, and the §6.1 microbenchmark, the §6.5 launch-amortization ablation arms, and the HUNL probe add minutes. Exploitability is exact NashConv via OpenSpiel's best response [33] — no sampled exploitability anywhere.